



University of
Nottingham

UK | CHINA | MALAYSIA

COMP2001/2011: Artificial Intelligence Methods

Lecture 4 – Move Acceptance and Parameter
Setting Issues

Dr Warren G Jackson



Announcements

- Assessment sheets are now approved* and available on Moodle.
- Lectures 5 and 6 will be delivered by Prof Ender Ozcan.

*Partially approved for COMP2001.

 **Module Information**

TEST whether you are in the right Moodle group (**COMP2001** or **COMP2011**) here.

The content and coursework assignments that you see on this Moodle page depends on you being in the correct Moodle group which should reflect your module enrolment. It is important that you check you are in the group that you expect by referring to the below assessment information. If you are in the wrong group, see both groups, or no groups at all, please email the module convenors to get this corrected.

Please note that being able to see your group as **COMP2001** or **COMP2011** does not reflect formal enrolment in the respective module - formal enrolment can be viewed on myNottingham.

If you switch between the 10 and 20 credit modules, please let the module convenors know so that they can update your Moodle group.

Module Assessment Information (COMP2001)

- [50%] Coursework (in-lab exams based on lab exercises and a project)
 - [20%] In-lab exam on the lab exercises and investigations.
 - [30%] Practical examination on the project during timetabled lab hours: The description for the project part of the coursework can be viewed in the "COMP2001 Project" tab below. (To be released in due time)
- [50%] (ExamSys) Exam

 Not available unless: You belong to **COMP2001** (hidden otherwise)

 [COMP2001 \(20-credits\) Assessment Sheet](#) 

Completion ▾

 Not available unless: You belong to **COMP2001** (hidden otherwise)

Module Assessment Information (COMP2011)

- [100%] Exam

 Not available unless: You belong to **COMP2011** (hidden otherwise)

 [COMP2011 \(10 credits\) Assessment Sheet](#) 

Completion ▾

 Not available unless: You belong to **COMP2011** (hidden otherwise)

The reassessment for this module is **100% examination**.

All marks are provisional as they are subject to the approval of examiners.



The landscape so far...

- Previously we have looked at:
 - Components of heuristic search
 - Simple “low-level” heuristics
 - Hill-climbing heuristics
 - Metaheuristics
 - Performance comparisons
- Today's Topics:
 - Move Acceptance Methods
 - Parameter Setting Issues
 - Methods for parameter tuning



Move Acceptance

Acceptance Strategies for Local Search Metaheuristics



Recap – General Search Template for Local Search Metaheuristics

INPUT: s_0 // starting solution

$s^* \leftarrow \text{initialise}(s_0)$ // could improve s_0 or use the original

REPEAT

$s' \leftarrow \text{makeMove}(s^*, \text{memory})$ // choose a neighbour s' of s^*

$\text{accept} = \text{moveAcceptance}(s^*, s', \text{memory})$ // decide to move to this neighbour or not

if (accept) $s^* \leftarrow s'$ // else reject and “ $s' \leftarrow s'$ ”

UNTIL (termination conditions are satisfied)

- Focus so far has been on how we make such neighbourhood moves.
- We have been rejecting worse cost moves...
- ...but maybe there are advantages to accepting these?



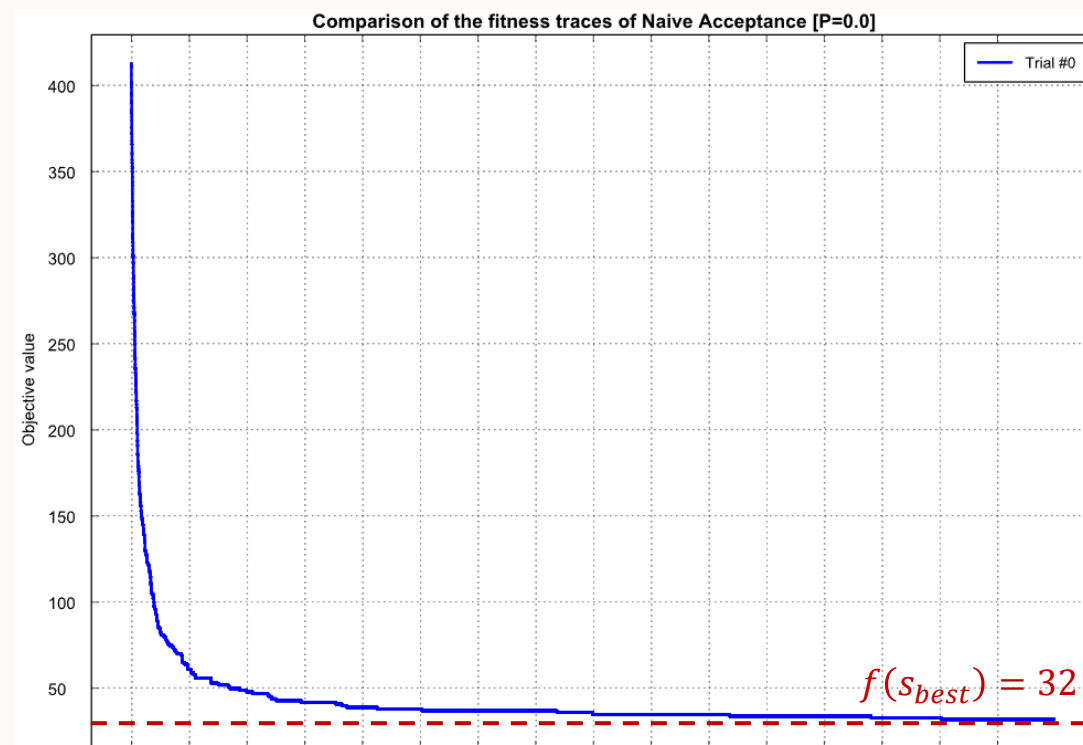
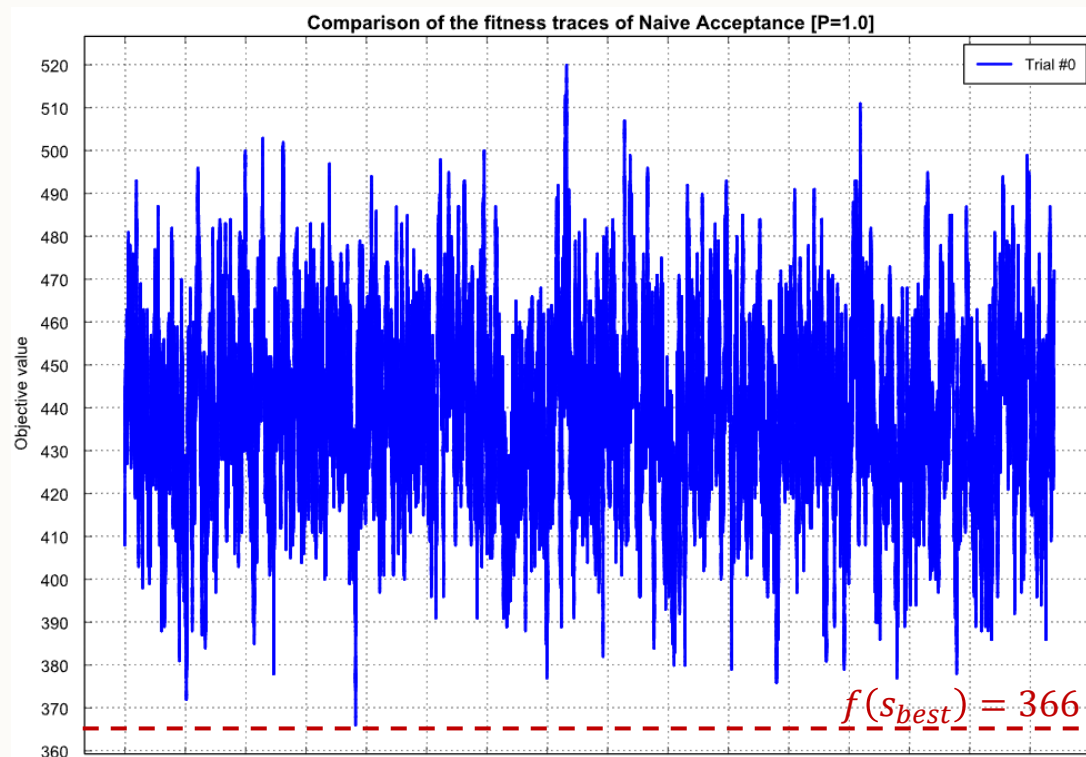
A Naïve Approach?

What happens if we accept some percentage of worst cost moves?

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

IF s' not worse_than s^* THEN return True

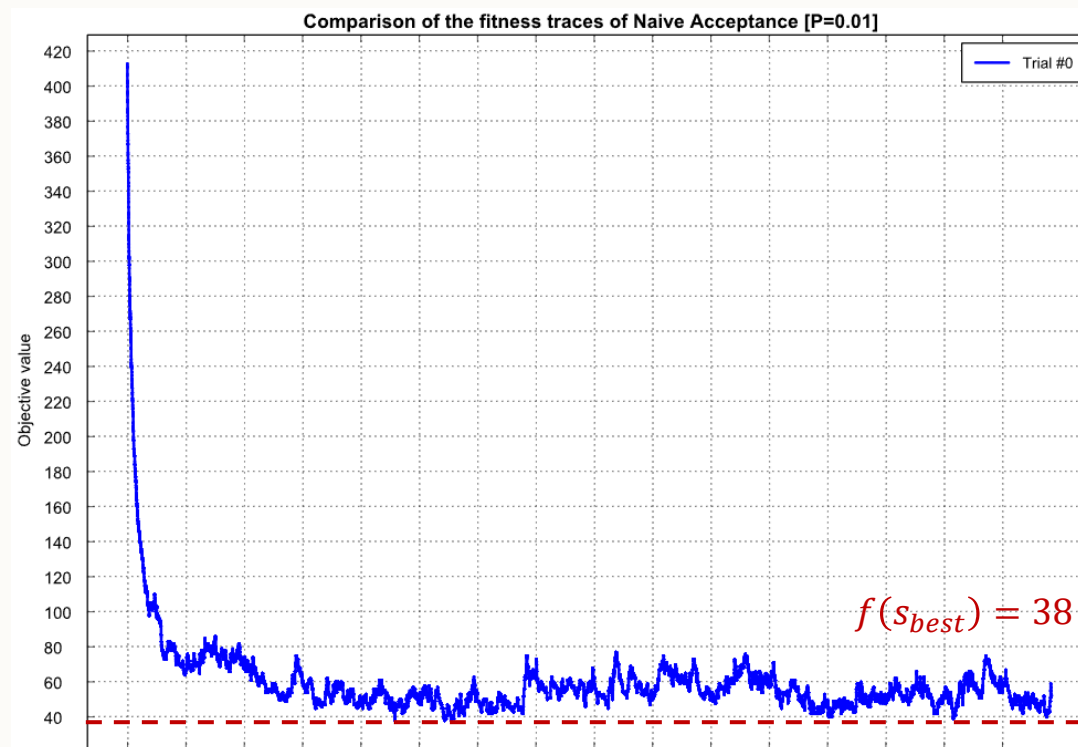
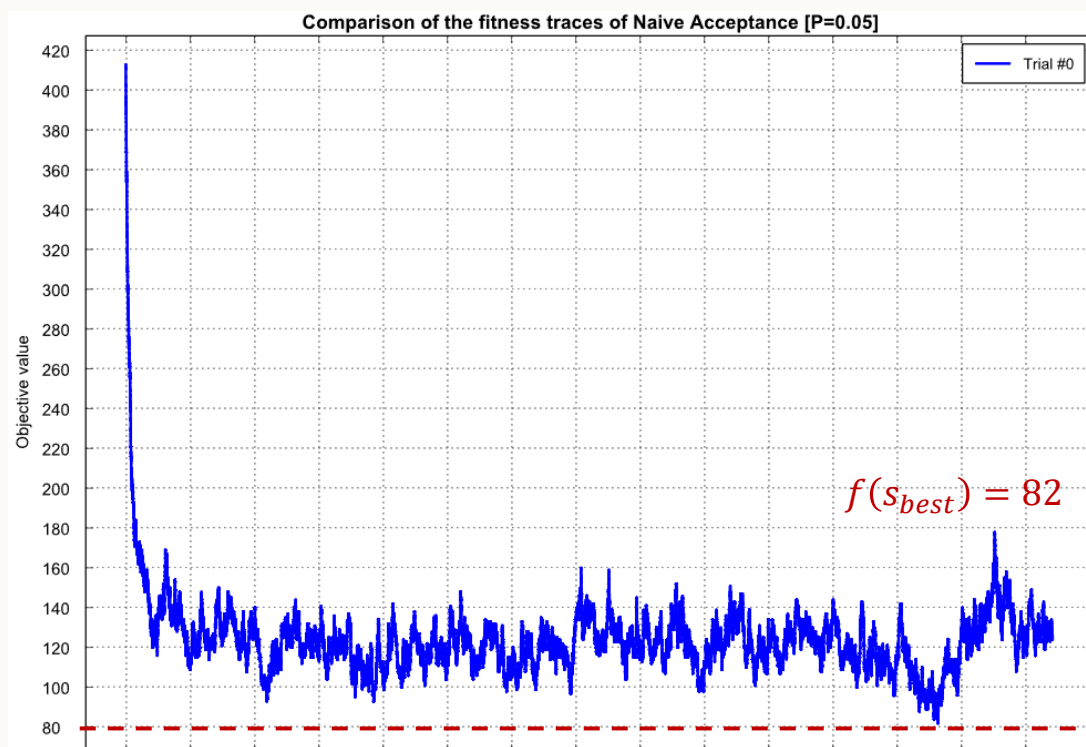
ELSE return random $\in [0,1) \leq P$





Naïve Acceptance – A Case for Move Acceptance?

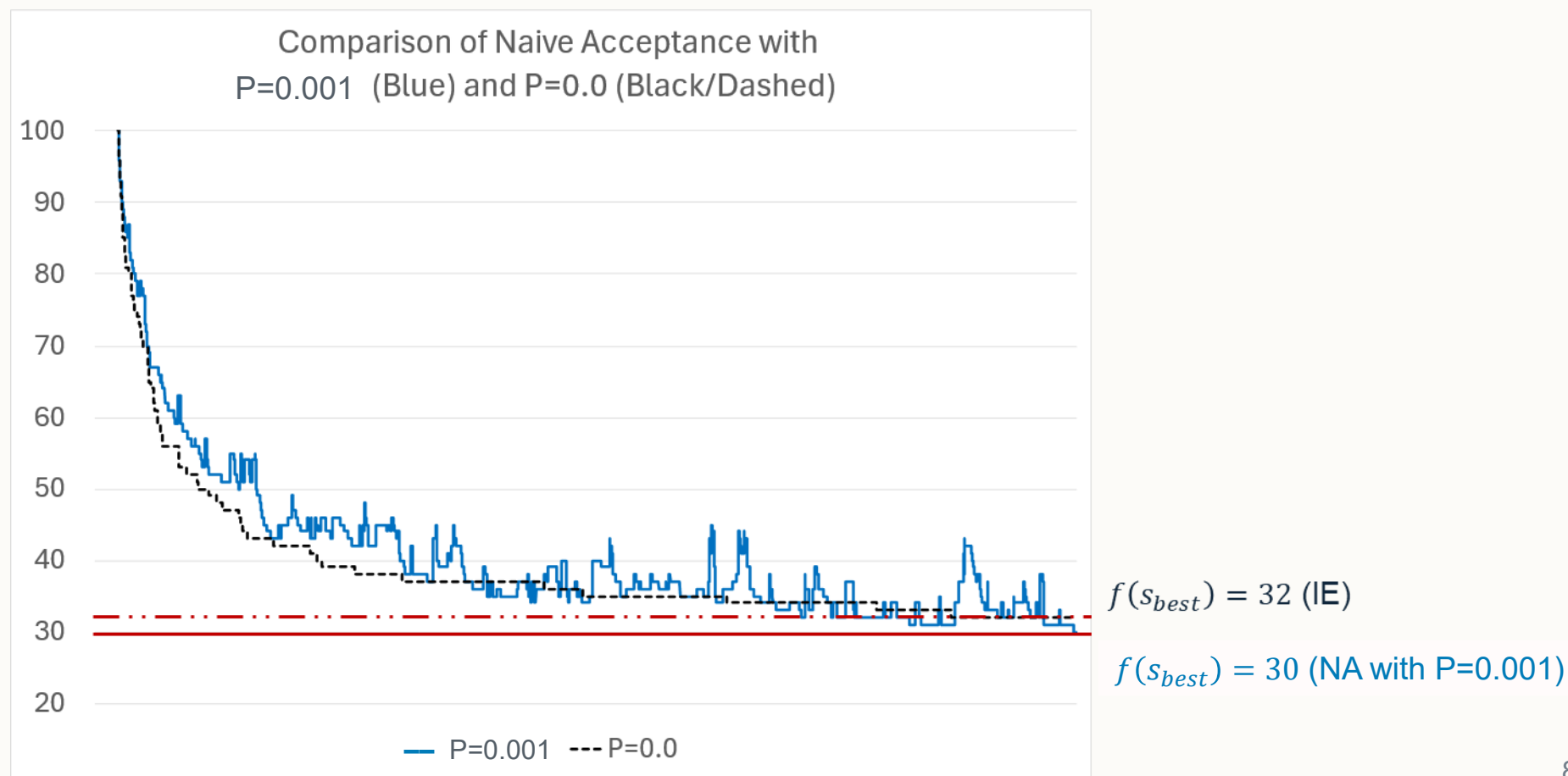
Changing the setting/configuration of P can significantly affect the performance of Naïve Acceptance.





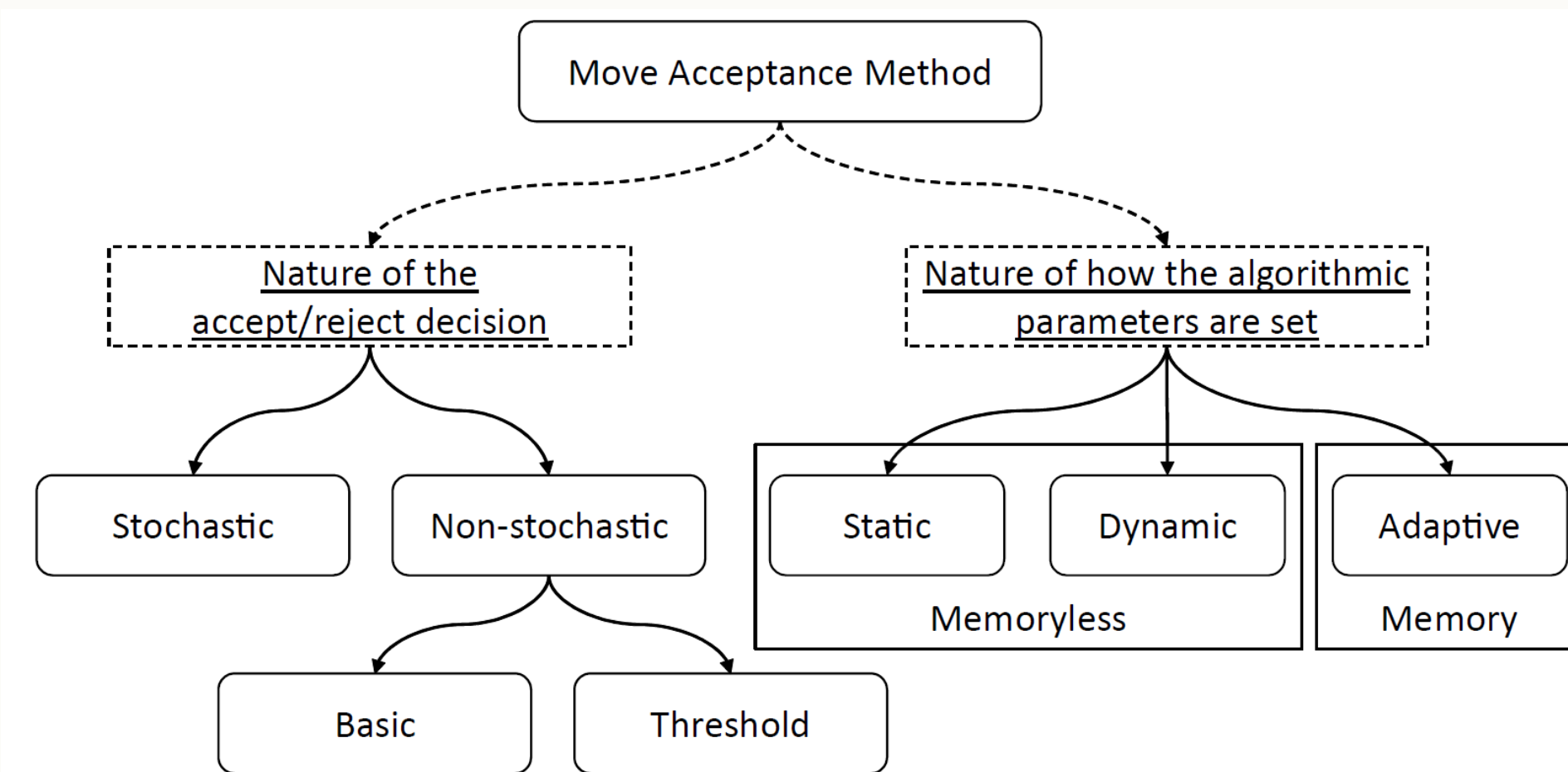
Naïve Acceptance – A Case for Move Acceptance?

Finding a good setting/configuration for P can be tedious, but with $P = 0.001$ we have been able to find a better solution than accepting non-worsening moves alone.





A Taxonomy for Move Acceptance Methods



W. G. Jackson, E. Özcan and R. I. John, Move Acceptance in Local Search Metaheuristics for Cross-domain Search, Expert Systems with Applications, 109: 131-151, 2018 [[original PDF](#)]. [[PDF](#)]



There Are Many Move Acceptance Methods

	Static	Dynamic	Adaptive
Non-stochastic Basic	AM (Cowling et al., 2001) IE (Bilgin et al., 2007) OI (Cowling et al., 2001)	-	LA (Burke and Bykov, 2008) AILLA-F (Mısır et al., 2011b) AILLA (Mısır et al., 2012) SCHC (Bykov and Petrovic, 2016)
Non-stochastic Threshold	(TA (Sabar et al., 2013))	TA (Dueck and Scheuer, 1990) GD (Dueck, 1993) (FD (Sin and Kham, 2012))	(TA (Tarantilis et al., 2004)) (GD (Sinclair, 1993)) FD (Burke and Bykov, 2006) EGD (McCollum et al., 2009) LBTA (Tarantilis and Kiranoudis, 2002) BATA (Tarantilis et al., 2004) RRT (Dueck, 1993) ILTA (Mısır et al., 2009) AILTA (Mısır et al., 2010)
Stochastic	EMC (Ayob and Kendall, 2003) LMC (Ayob and Kendall, 2003) NA (Burke et al., 2010)	(EMC (Özcan et al., 2008)) SA (Kirkpatrick et al., 1983)	EMCQ (Ayob and Kendall, 2003) (SA (Ingber, 1989)) SARH (Connolly, 1992) ¹ NLGD (Landa-Silva and Obit, 2008) AA (Burke et al., 2010)



Algorithmic Parameter Settings

- Many move acceptance methods are parameterised.
- The way parameters are set/controlled vary and is captured in the previous taxonomy:
 - **Static** methods behave the same throughout the search. Either they contain no parameters, or the parameters are fixed.
 - **Dynamic** methods may behave differently in time but this change in behaviour is predetermined. The values of parameters change in time and depend on the current (iteration/time).
 - **Adaptive** methods may behave differently based upon the search (history/memory).
- Examples: given two solutions s_i^* and s_i' , s_i' may or may not be accepted based on $moveAcceptance(s_i^*, s_i', memory)$ in each setting method...



Nature of Acceptance Decision

- Concerns the way in which values are determined within the move acceptance method.
- The objective value of a candidate solution can be directly compared to known solution values (**basic**) or other calculated values (**threshold**) not necessarily equivalent to the cost of a solution, or acceptance is in some way determined based on a randomised probability/component and accepted in a non-deterministic way (**stochastic**).
- Examples:
 - $f(s'_i) \leq f(s_{i-5}^*)$
 - $f(s'_i) \leq f(s_i^{best}) + 10$
 - $rand(0,1) \leq e^{-\frac{f(s_i^*) - f(s'_i)}{T}}$



Move Acceptance So Far

- So far, we have only been using the **static non-stochastic basic** criteria to accept neighbourhood moves:
 - All Moves (AM) acceptance always accepts/returns true.
 - Improving or Equals (IE): $f(s') \leq f(s)$
 - Also called non-worsening
 - Only Improving (OI): $f(s') < f(s)$



Late Acceptance – Overview

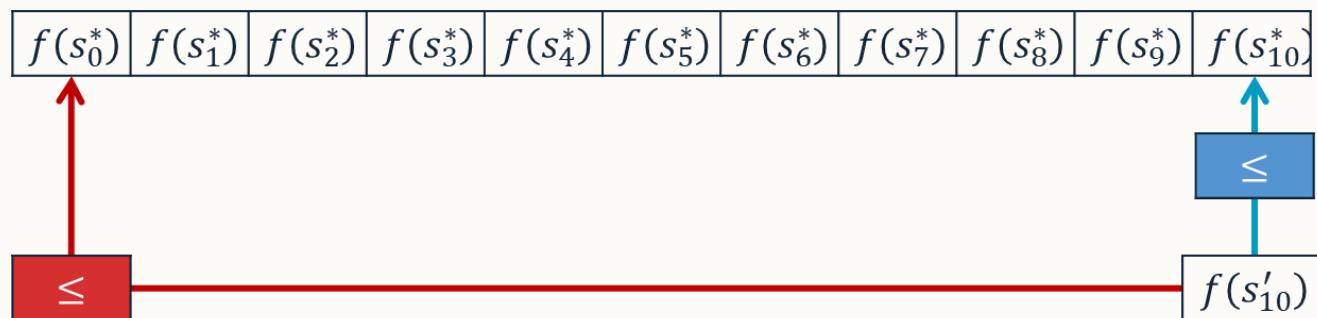
- A **basic adaptive** move acceptance method accepting worse cost moves if not worse than the solution accepted L steps previous.

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

IF $f(s') \leq f(s^*)$ THEN return True ■

ELIF $f(s'_i) \leq f(s_{i-L}^*)$ THEN return True ■

ELSE return False





Late Acceptance – Initialisation

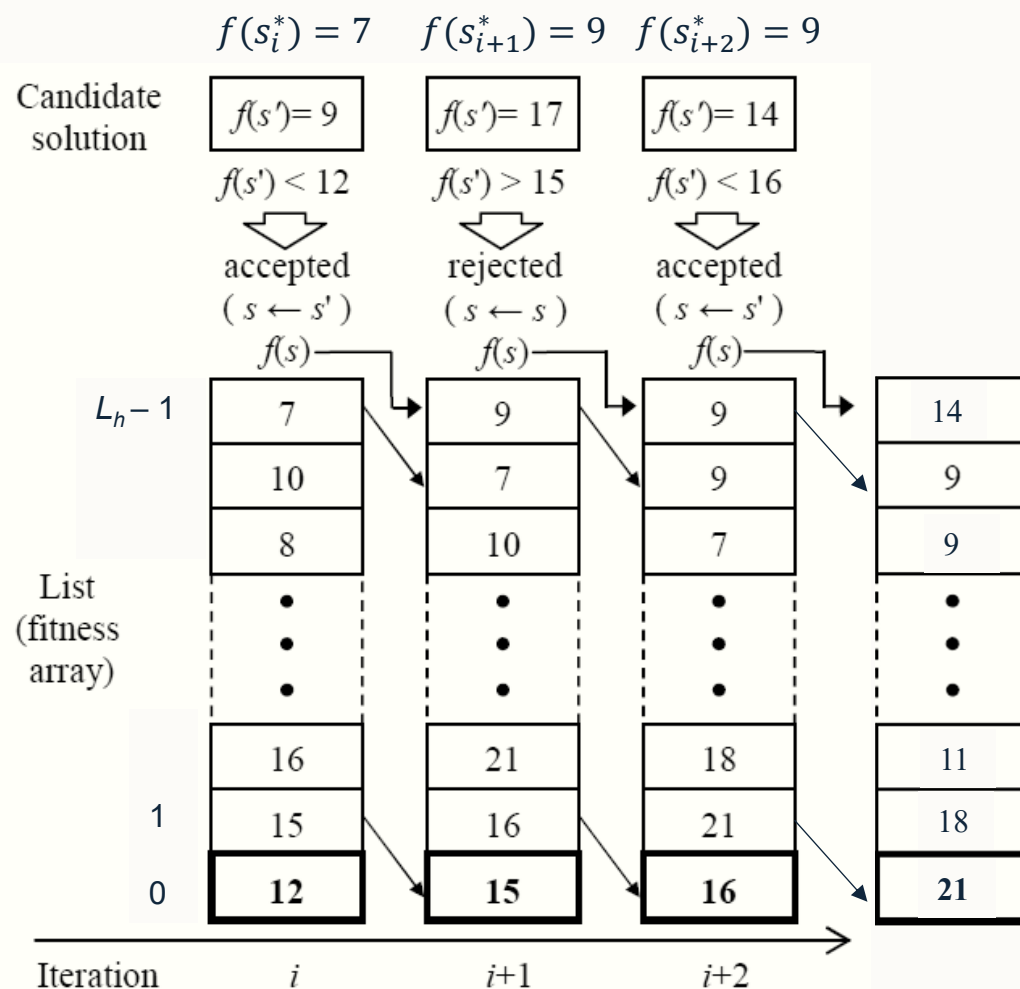
- Late Acceptance contains a list of length L previous accepted solution costs.
- Which costs to use upon first L steps?

Can initialise the list with:

- Initial solution cost – most common.
- Infinity (random walk for L steps).
- Zero (force hill-climbing to begin with).



Using Late Acceptance for Search





Will the candidate solution be accepted using Late Acceptance given the state defined on the slides?

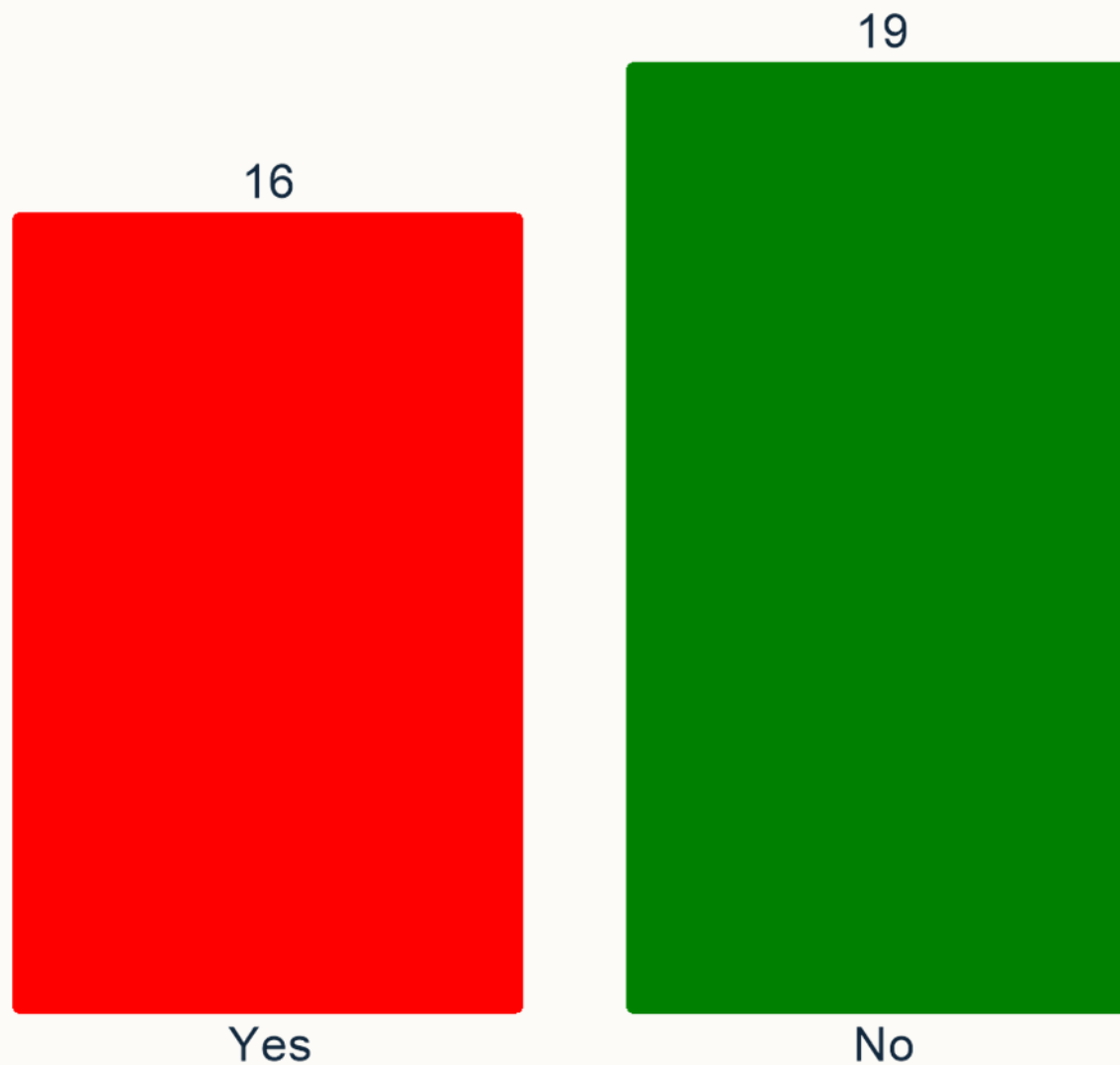
A. Yes

✓ B. No

Given that:

- $f(s_{10}^*) = 50$
- $f(s'_{10}) = 60$
- $L = 3$

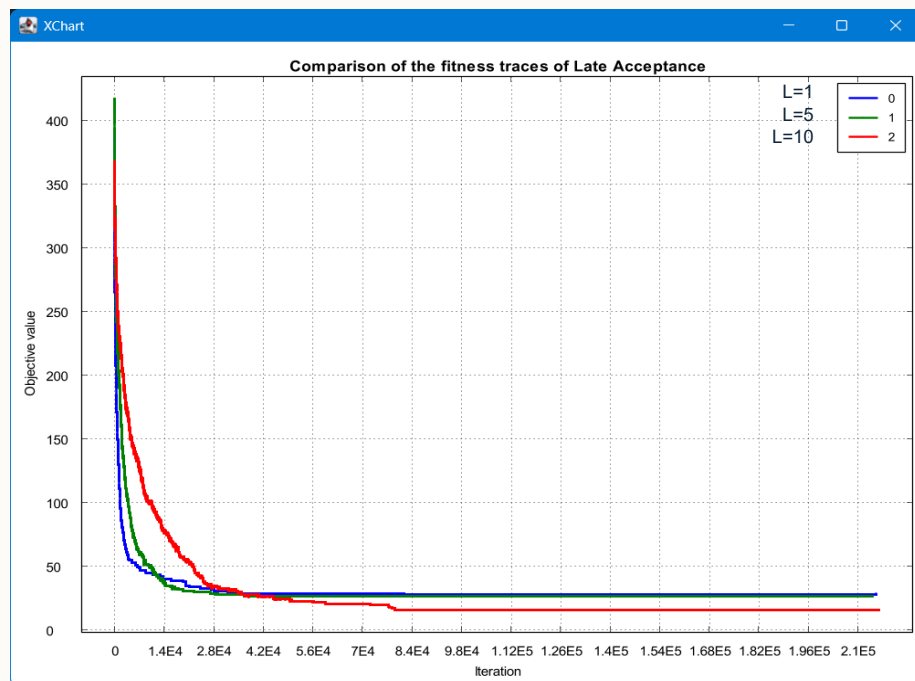
$f(s_9^*)$	60
$f(s_8^*)$	65
$f(s_7^*)$	55
$f(s_6^*)$	60
$f(s_5^*)$	62
$f(s_4^*)$	65





Effects of List Length on the Search

- Setting of the list length vital on search performance.
- Short list length has fast convergence with sub-optimal results.
- Longer list length has slow convergence with better results generally.





Non-stochastic Threshold Methods

- **Non-stochastic threshold** criteria determines acceptance of neighbourhood moves based on a comparison with some calculated value (may not correspond to an objective value of a solution):

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

`$T \leftarrow \text{updateThresholdValue}(s', \text{memory})$`

`IF  $f(s') \leq \max(f(s_i^*), T)$  THEN return True`

`ELSE return False`

- This threshold could be based on a combination of an ε value with the cost of the (initial/current/best-found-so-far/stage-best) solution.



Non-stochastic Threshold Methods – Nature of the Parameter Settings

- **Static** methods can only use a fixed ε value in relation to the cost of the current solution.
 - Threshold Acceptance with a fixed ε .
- **Dynamic** methods generally decrease an ε value over time towards a known target.
 - Great Deluge (Example in later slides).
- **Adaptive** methods may adapt an ε value over time and combine this with memory to determine a threshold.
 - Record-to-record Travel – where $T = \varepsilon + f(s_i^{best})$
 - Extended Great Deluge
 - Flex Deluge



Great Deluge – Overview

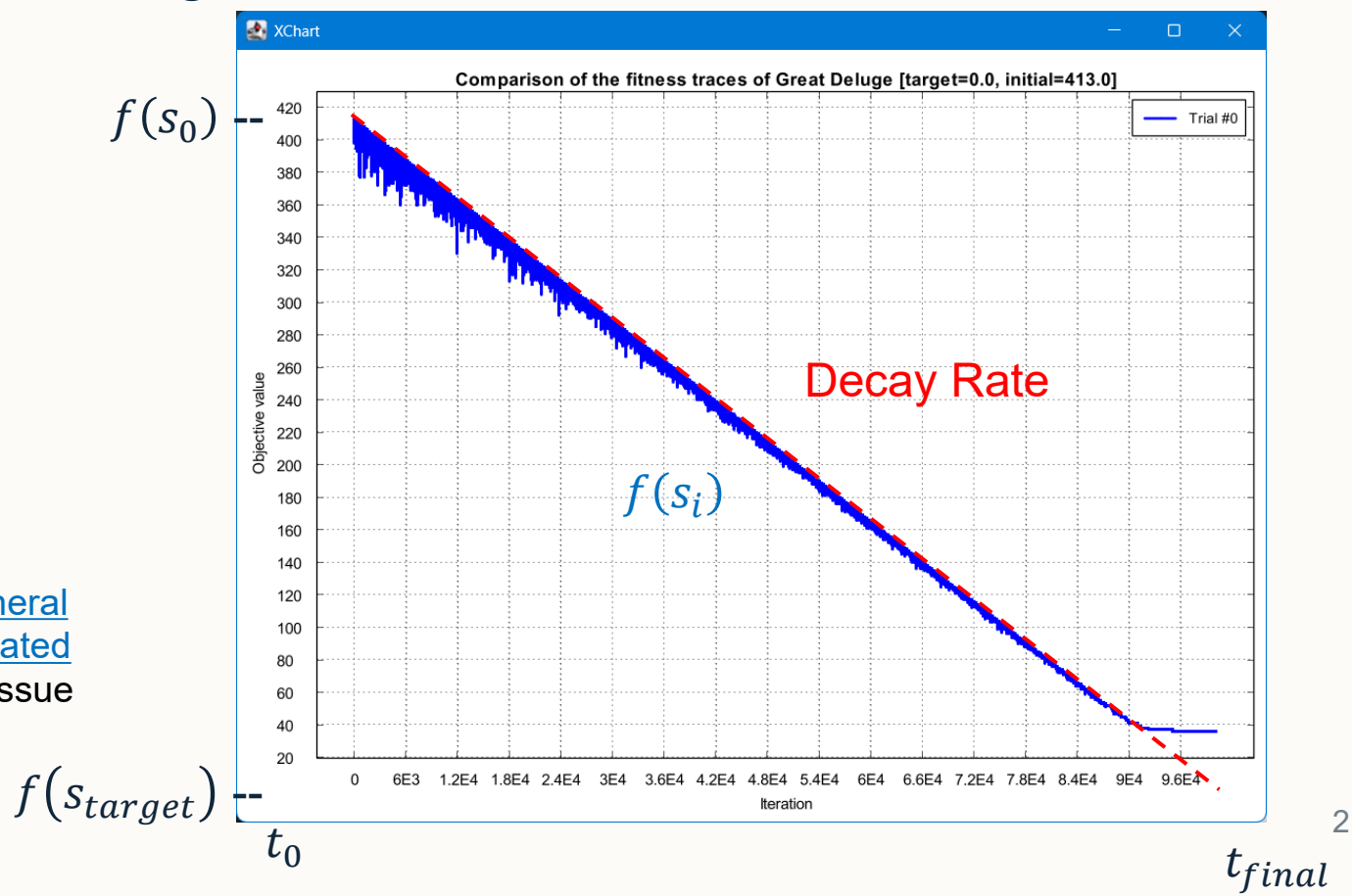
- A **dynamic non-stochastic threshold** move acceptance method accepting worse cost moves if not worse than a “water-level” that decreases towards a specified target over time.

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

$T = T - \text{DecayRate}$

`return  $f(s') \leq \max(f(s_i^*), T)$`

Gunter Dueck, Tobias Scheuer, [Threshold accepting: A general purpose optimization algorithm appearing superior to simulated annealing](#), Journal of Computational Physics, Volume 90, Issue 1, September 1990, Pages 161-175

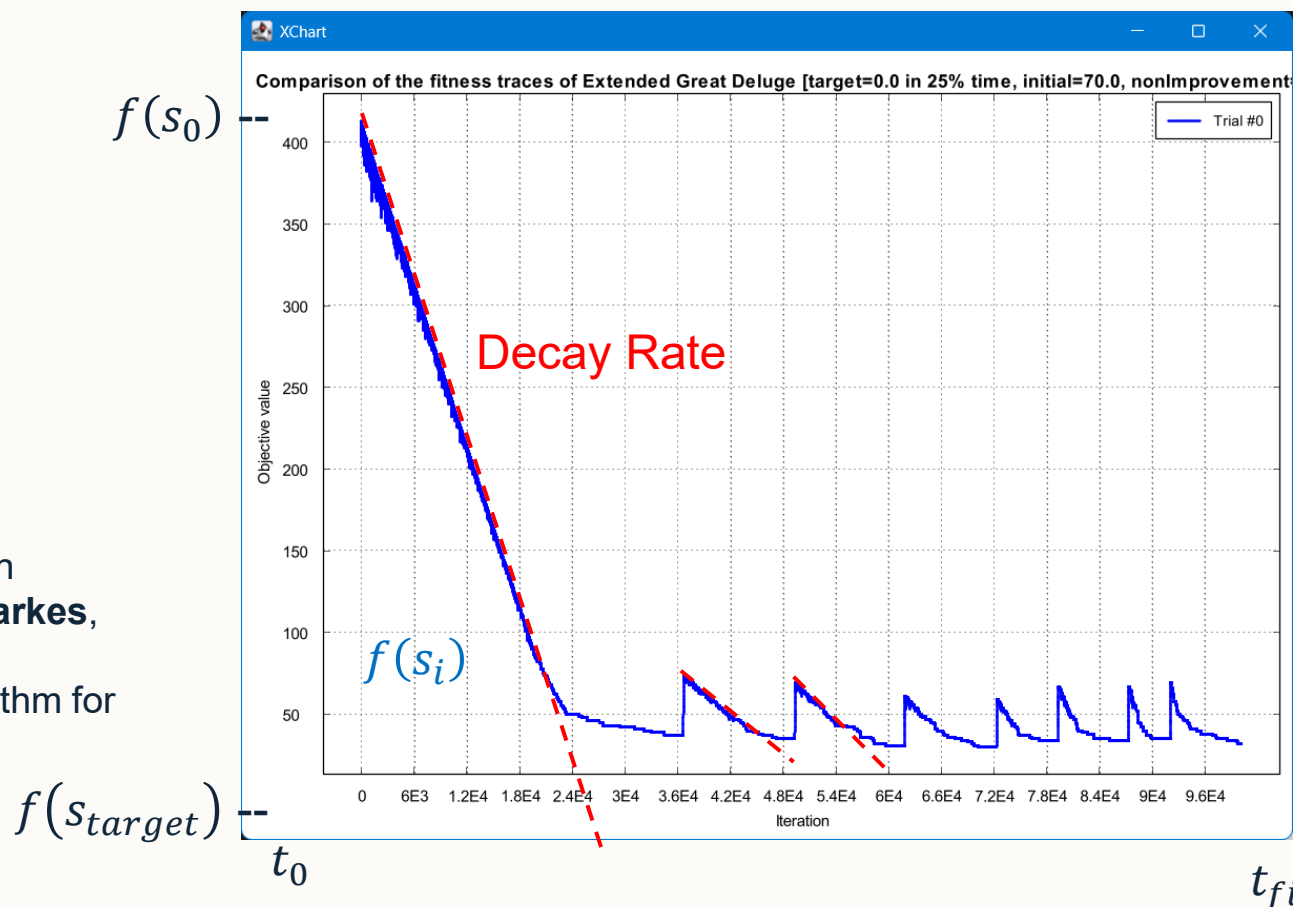




Extended Great Deluge – Overview

- An **adaptive non-stochastic threshold** move acceptance method extending Great Deluge by updating the water level and decay rate based on feedback from the search.
- Key idea is to allow restarts when the search is thought to be stuck.

[1] An Extended Great Deluge Approach to the Examination Timetabling Problem, B. McCollum, P.J. McMullan, **A. J. Parkes**, E.K. Burke, S. Abdullah [\[PDF\]](#)
[2] An Extended Implementation of the Great Deluge Algorithm for Course Timetabling, Paul McMullan [\[PDF\]](#)





Stochastic Methods

- **Stochastic** criteria determines acceptance of neighbourhood moves based on a randomised component.

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

`$p \leftarrow \text{random}[0,1)$`

`IF  $f(s') \leq f(s_i^*)$  THEN return True`

`IF  $p < g(s', s_i^*, \text{memory})$  THEN return True`

`ELSE return False`

Note randomly adjusting internal parameters of any of the methods seen so far puts them into the stochastic category.



Stochastic Methods – Nature of the Parameter Settings

- **Static** methods use a fixed probability, P .
 - Naïve Acceptance – with a fixed P .
- **Dynamic** methods adjust the probability over time.
 - Simulated Annealing – (Example in later slides) dynamically decreases a temperature parameter over time.
- **Adaptive** methods adapt the probability based on feedback from the search.
 - Simulated Annealing with Reheating – increases/decreases the temperature setting to balance exploitation/exploration.



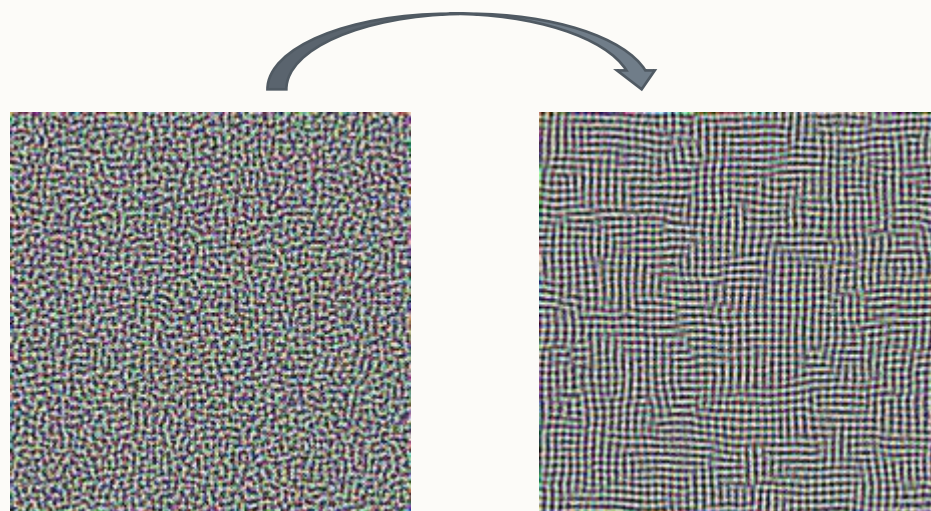
Simulated Annealing – Background

- A stochastic local search algorithm inspired by the physical process of annealing (Kirkpatrick et al. 1983).
- Easy to implement.
- Achieves good performance given sufficient running time.
- Requires a good parameter setting for improved performance.
- Has interesting theoretical properties (convergence), but these are of very limited practical relevance for optimisation.



Simulated Annealing – The Analogy

- If a liquid material **cools and anneals** too quickly, then the material will solidify into a **sub-optimal** configuration.
- If the liquid material **cools slowly**, the crystals within the material will solidify **optimally into a state of minimum energy** (i.e. ground state).
 - This ground state corresponds to the minimum of the cost function in an optimisation problem.





Simulated Annealing – Overview

- A **dynamic stochastic** move acceptance method that accepts worse cost moves based on a probability that decreases both over time and based on the measure of change.

`moveAcceptance( $s^*$ ,  $s'$ , memory):`

`$r \leftarrow \text{random}[0,1)$`

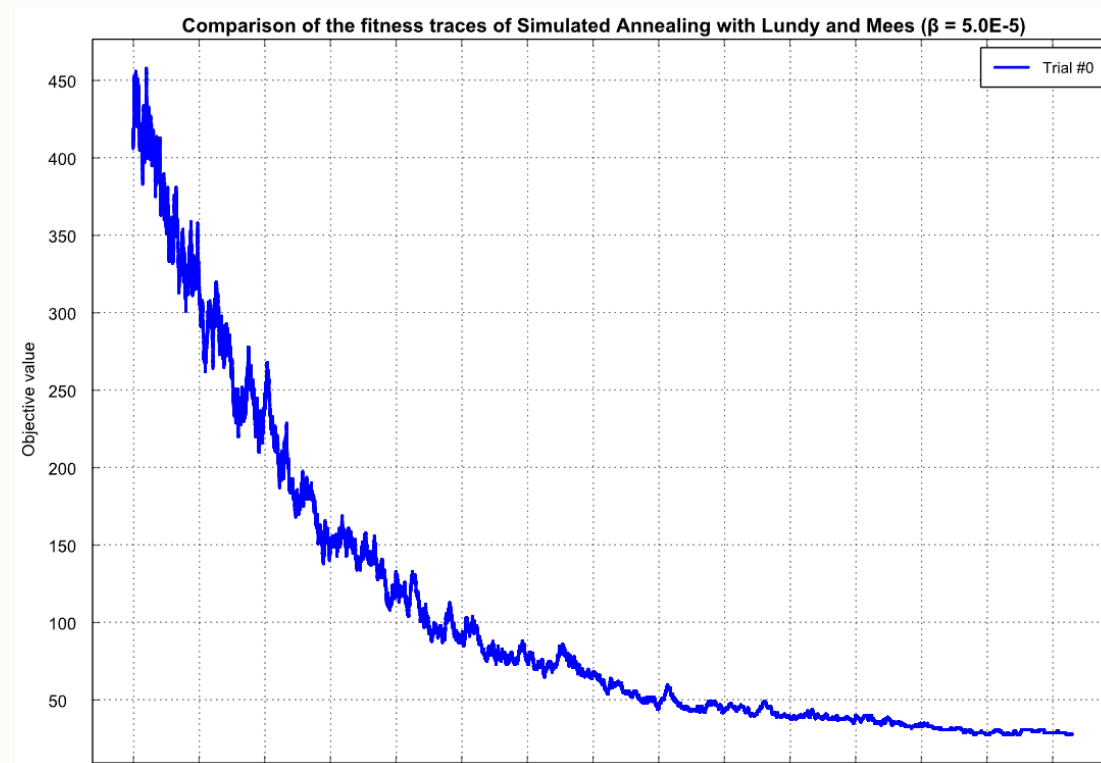
`$\Delta = f(s') - f(s^*)$`

`acceptance =  $\Delta < 0 \parallel r < P(\Delta, T)$`

`T = coolTemperature()`

`return acceptance`

- P is the Boltzmann probability.





Accepting Moves Using Simulated Annealing

- Improving moves are always accepted ($\Delta < 0$).
- Non-improving moves accepted using the Boltzmann probability:
 - $P(\Delta, T) = e^{\left(-\frac{\Delta}{T}\right)}$
 - Accept if our random number is less than this value.

```
moveAcceptance(s*, s', memory):  
    r ← random[0,1)  
    Δ = f(s') − f(s*)  
    acceptance = Δ < 0 || r < P(Δ, T)  
    T = coolTemperature()  
    return acceptance
```



Is Simulated Annealing guaranteed to accept equal cost moves (when $\Delta = 0$)?

✓ A. Yes

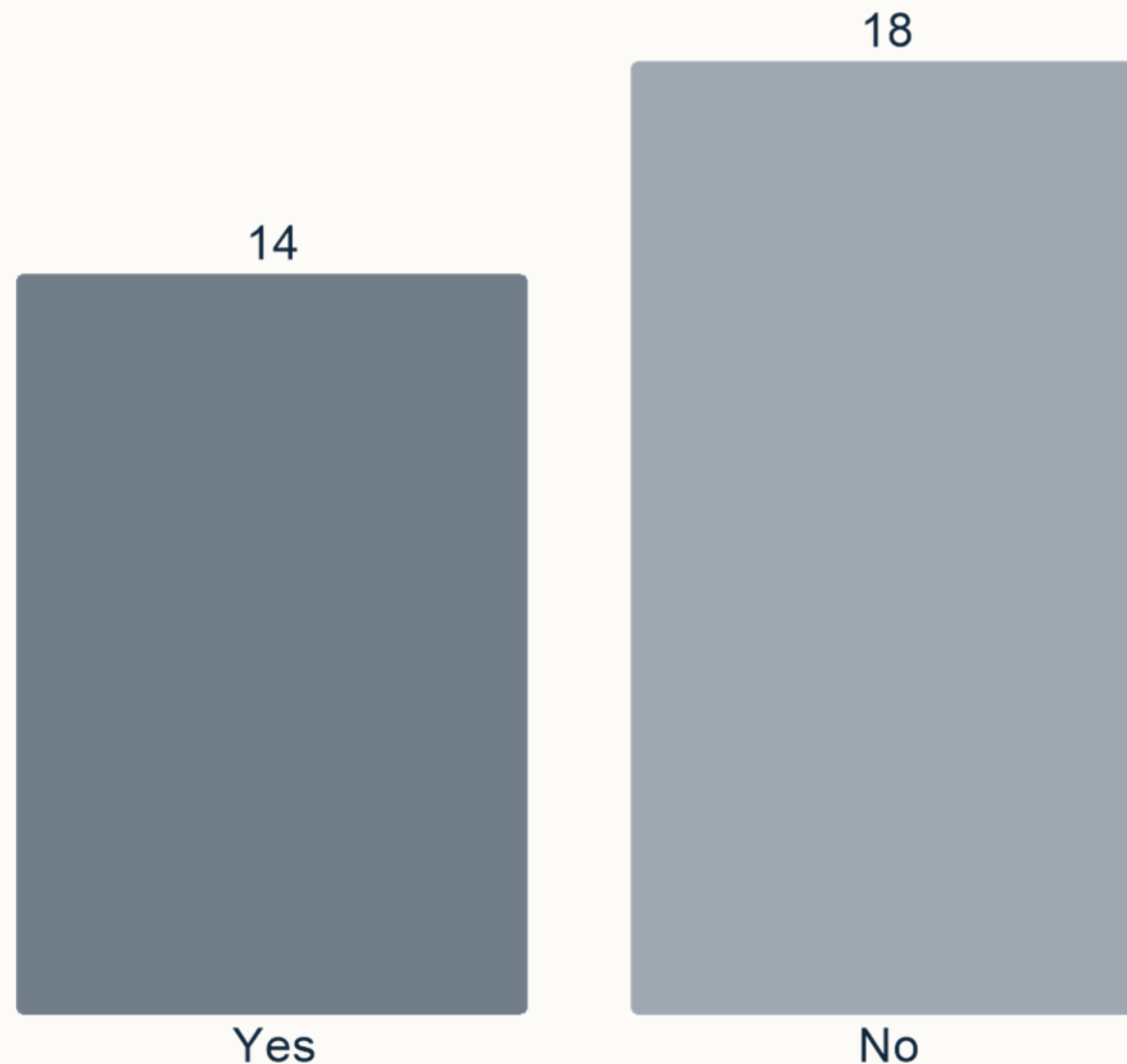
B. No

Improving moves are always accepted ($\Delta < 0$).
Non-improving moves accepted using the Boltzmann probability:

$$P(\Delta, T) = e^{\left(-\frac{\Delta}{T}\right)}$$

Accept if our random number
is less than this value.

```
moveAcceptance(s*, s', memory):  
    r ← random[0,1)  
    Δ = f(s') - f(s*)  
    acceptance = Δ < 0 || r < P(Δ, T)  
    T = coolTemperature()  
    return acceptance
```





Effects of Cooling (Annealing)

- Through time, the temperature of the system is decreased to a ground state.
- A high temperature allows a higher magnitude of inferior moves to be accepted.
- A low temperature rejects inferior moves, accepting a few minor worse cost moves.
- Decreasing T decreases the probability that a given worse cost move is accepted.
- For a given T , we can also observe the probability decreases as Δ increases.

	Probability of acceptance $P(\Delta, T) = e^{-\Delta/T}$		
T	$\Delta=1$	$\Delta=2$	$\Delta=3$
10	0.9	0.82	0.74
1	0.37	0.14	0.05
0.25	0.018	0.0003	0.000006
0.1	0.00005	2×10^{-9}	9×10^{-14}



Cooling Schedules for Simulated Annealing

- There are many ways in which we can decrease the temperature, each method is a specific cooling schedule.
- Cooling schedules usually specify:
 - An initial/starting temperature, T_0
 - A final temperature T_{final}
 - A mechanism for decreasing the temperature
 - The number of iterations per temperature setting.
- Changing any of these components can significantly affect the performance!



Cooling Schedules – Starting Temperature

- T_0 needs to be hot enough to allow most worse cost moves, but not too hot that we perform random walk.
- Initial temperature settings can be calculated using computational methods to accept a target percentage of worse moves.
 - Needs to be done for every problem instance.
 - An example can be seen in [Computing the Initial Temperature of Simulated Annealing | Computational Optimization and Applications](#)



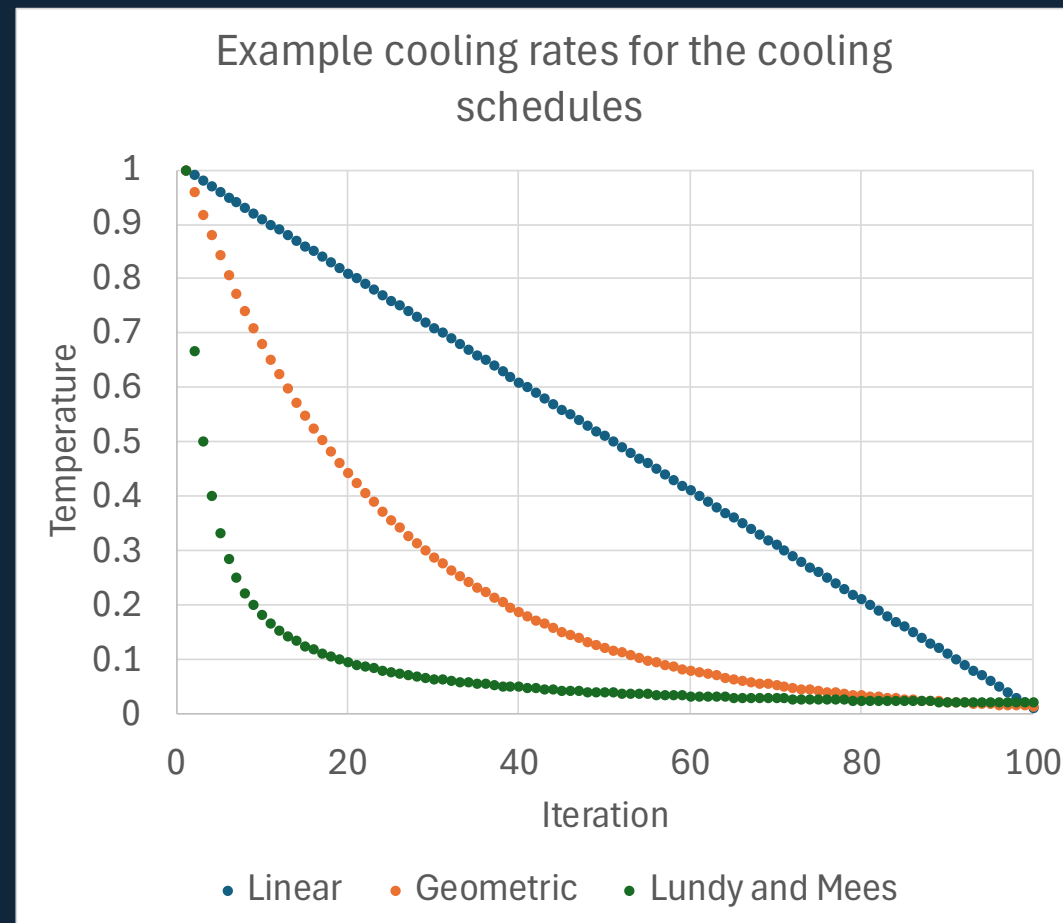
Cooling Schedules – Final Temperature

- Usually assumed to be (close to) 0
- Should be low enough such that there is close to zero chance a worse cost move is accepted near to the end of the search.
- Parameter(s) of temperature decrement mechanism can be specified to reach this value (or vice versa).



Cooling Schedules – Decrement Mechanisms

- **Linear** cooling reduces the temperature by a fixed amount each iteration
 - $T_{i+1} = T_i - T_\delta$
- **Geometric** cooling reduces the temperature by multiplying the current temperature by a value (α) close to 1 and **usually** $\alpha = [0.9, 0.99]$ but is problem instance specific.
 - $T_{i+1} = T_i \cdot \alpha$
- **Lundy and Mees** cooling somewhat similar, uses a parameter β close to 0 and **usually** around 0.0001.
 - $T_{i+1} = \frac{T_i}{1 + \beta T_i}$





Cooling Schedules – Iterations per Temperature

- Standard approach is to decrease the temperature after each iteration.
- Other mechanisms exist:
 - Reduce T after some fixed number of iterations.
 - Reduce T after some number of iterations that is changed based on where we are in the search (reduce often to begin with but slow the rate of change towards the end).
- **Adaptive** approaches:
 - Reduce T after some number of accepted moves.
 - Simulated Annealing with **Reheating** increases the temperature at certain points based on feedback.



Exercise – Using Simulated Annealing for Solving a MAX-SAT Problem (Step 1)

▪ Given:

- $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
- $T_0 = 0.9; \alpha = 0.9$ – Geometric Cooling
- $s_0 = \langle 100100 \rangle$ with $f(s_0) = 3$
- $randomInts = [2,3,1,6]; randomFloats = [0.470, 0.253]$

▪ Step 1 (i=1)

- $s'_1 = \langle 110100 \rangle$ with $f(s'_1) = 4$
- $\Delta = f(s'_1) - f(s_0) = 4 - 3 = 1$
- $P(\Delta, T) = e^{-1/0.9} = 0.329$
- $accept = \Delta < 0 \parallel random < P(\Delta, T) = 1 < 0 \parallel 0.470 < 0.329$
- $s_2 \leftarrow s_1$
- $T_1 = T_0 \cdot \alpha = 0.9 \cdot 0.9 = 0.81$



Exercise – Using Simulated Annealing for Solving a MAX-SAT Problem (Step 2)

▪ Given:

- $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
- $T_1 = 0.81$; $\alpha = 0.9$ – Geometric Cooling
- $s_1 = \langle 100100 \rangle$ with $f(s_1) = 3$
- $randomInts = [\dots, 3, 1, 6]$; $randomFloats = [\dots, 0.253]$

▪ Step 2 (i=2)

- $s'_2 = \langle 101100 \rangle$ with $f(s'_2) = 1$
- $\Delta = f(s'_2) - f(s_1) = 1 - 3 = -2$
- $P(\Delta, T) = e^{-\Delta/T} = e^{-(-2)/0.81} = 1$
- $accept = \Delta < 0 \parallel random < P(\Delta, T) = -2 < 0$
- $s_3 \leftarrow s'_2$
- $T_2 = T_1 \cdot \alpha = 0.81 \cdot 0.9 = 0.729$

← Notice there was no need to throw a random number since Δ was less than 0.



Exercise – Using Simulated Annealing for Solving a MAX-SAT Problem (Step 3)

- Given $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
 - $T_3 = 0.729$; $\alpha = 0.9$ – Geometric Cooling
 - $s_2 = \langle 101100 \rangle$ with $f(s_2) = 1$
 - $randomInts = [\dots, 1, 6]$; $randomFloats = [\dots, 0.089]$
- Step 3 (i=3)
 - $s'_3 = \langle 111100 \rangle$ with $f(s'_3) = 2$
 - $\Delta = f(s'_3) - f(s_3) = 2 - 1 = 1$
 - $P(\Delta, T) = e^{-1/0.729} = 0.253$
 - $accept = \Delta < 0 \parallel random < P(\Delta, T) = 1 < 0 \parallel 0.089 < 0.253$
 - $s_4 \leftarrow s'_3$
 - $T_3 = T_2 \cdot \alpha = 0.729 \cdot 0.9 = 0.6561$



Complete step 3 and indicate if the candidate solution should be accepted or rejected using Simulated Annealing as on the slide?

Given:

$$\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$$

$$T_3 = 0.729; \alpha = 0.9 - \text{Geometric Cooling}$$

$$s_2 = \langle 101100 \rangle \text{ with } f(s_2) = 1$$

$$\text{randomInts} = [\dots, 1, 6]; \text{randomFloats} = [\dots, 0.253]$$

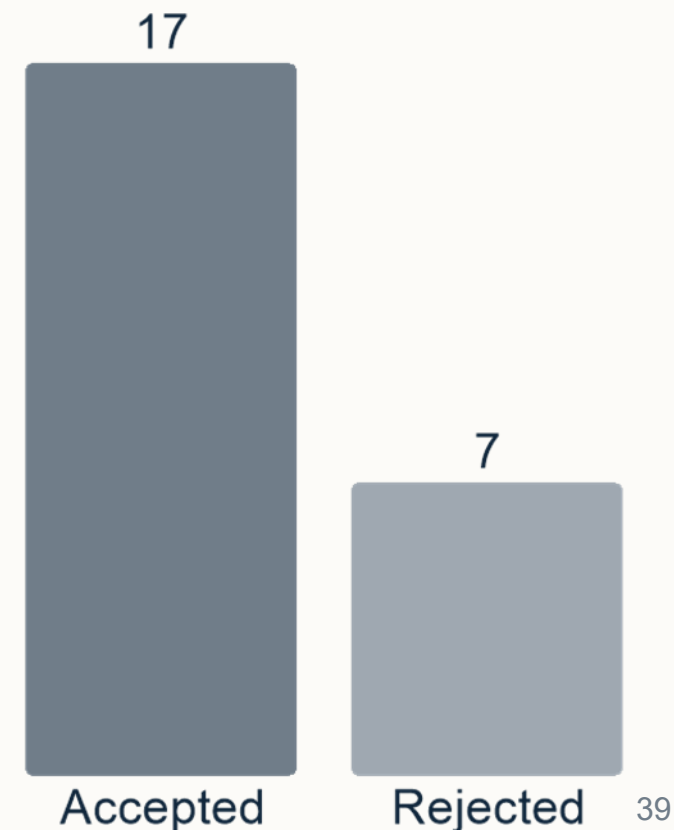
Step 3 (i=3)

$$s'_3 = \langle 111100 \rangle \text{ with } f(s'_3) = 2$$

...

A. Accepted

B. Rejected





Exercise – Using Simulated Annealing for Solving a MAX-SAT Problem (Step 3)

▪ Given:

- $\varphi = (x_0 \vee x_1) \wedge (\neg x_3 \vee x_5) \wedge (\neg x_0 \vee x_2) \wedge (x_1 \vee \neg x_5) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_4)$
- $T_3 = 0.729$; $\alpha = 0.9$ – Geometric Cooling
- $s_2 = \langle 101100 \rangle$ with $f(s_2) = 1$
- $randomInts = [\dots, 1, 6]$; $randomFloats = [\dots, 0.253]$

▪ Step 3 (i=3)

- $s'_3 = \langle 111100 \rangle$ with $f(s'_3) = 2$
- $\Delta = f(s'_3) - f(s_3) = 2 - 1 = 1$
- $P(\Delta, T) = e^{-1/0.729} = 0.253$
- $accept = \Delta < 0 \parallel random < P(\Delta, T) = 1 < 0 \parallel 0.253 < 0.253$
- $s_4 \leftarrow s_3^*$
- $T_3 = T_2 \cdot \alpha = 0.729 \cdot 0.9 = 0.6561$

Note that here truncated the calculation of the Boltzmann probability to 0.253 hence this fails. If we rounding or used full precision, then $e^{-\frac{\Delta}{T}} = 0.25366$ which would pass and accept the solution.



Hybrid Strategies for Move Acceptance

- In **single-stage** search methods, **group decision making** can be used to combine the acceptance decisions of multiple other move acceptance methods.
 - Acceptance can then be based on the majority vote.
- In **multi-stage** search methods, the process is broken down into multiple distinct phases.
 - Can use entirely different move acceptance methods dependant on the current stage.
 - E.g. may have multiple sequential stages of exploration-exploitation.



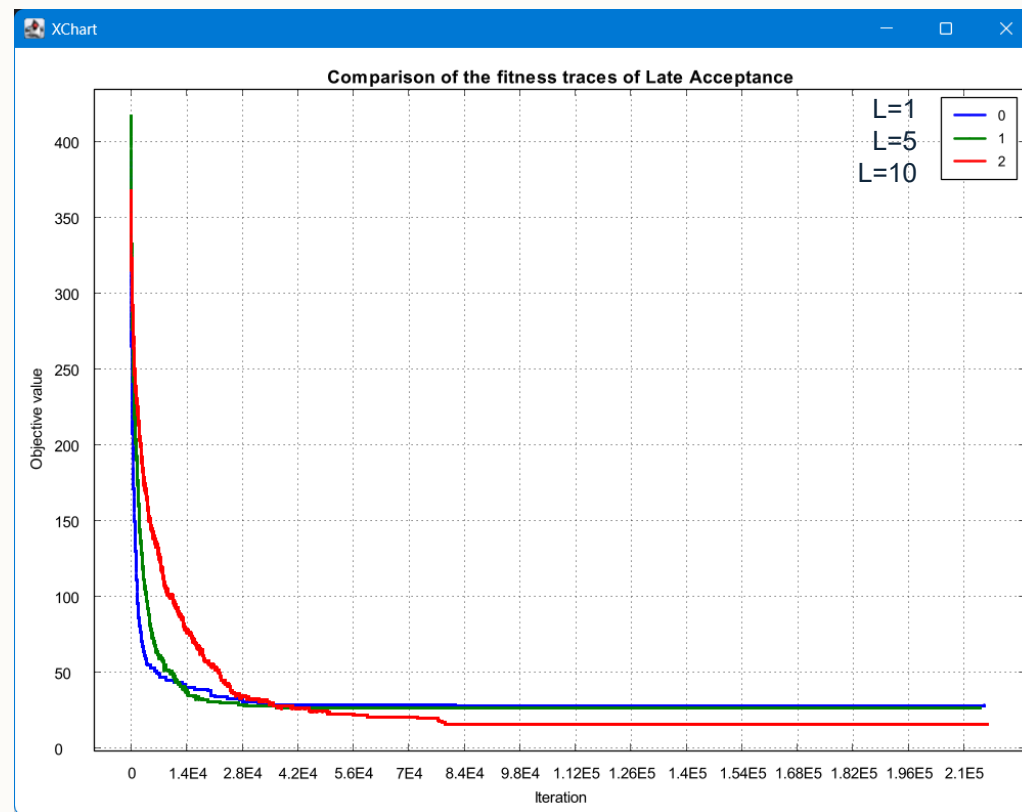
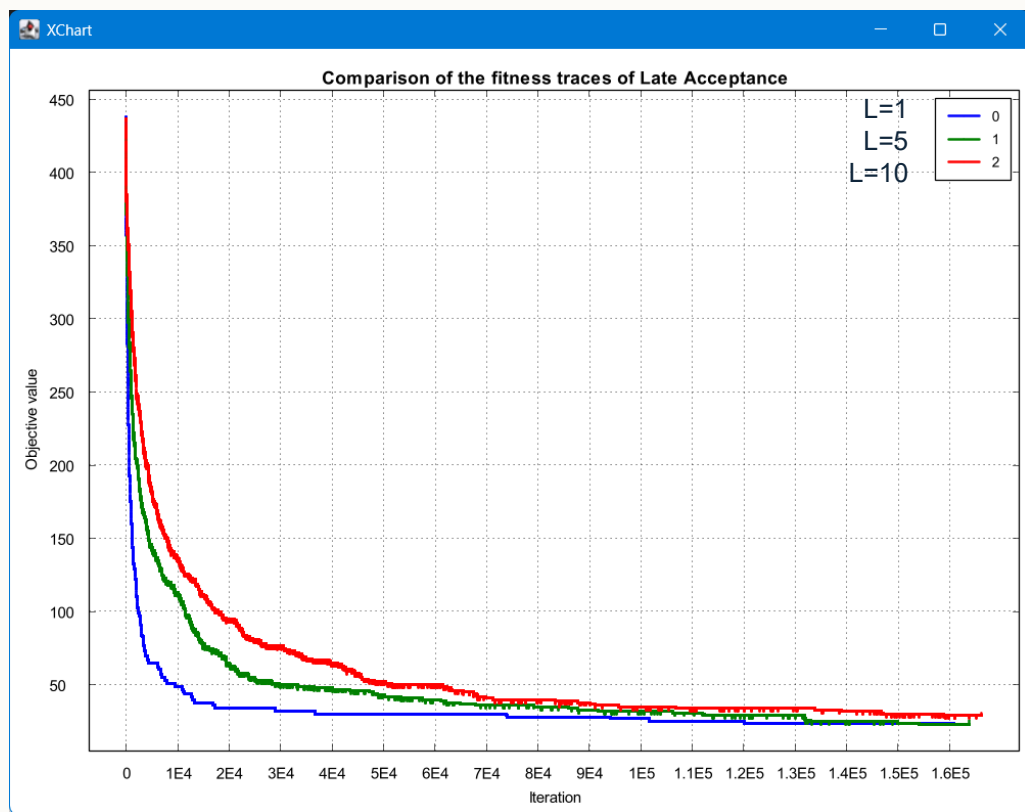
Parameter Configuration

Techniques for parameter setting/tuning/configuration



Heuristic Techniques are Sensitive to their Parameter Settings

- Take the following example of Late Acceptance solving two different MAX-SAT problem instances with 10 seconds of run time:





Most (Meta/Hyper) Heuristics and Move Acceptance Methods contain Parameters

- Examples of parameters in the techniques we have seen so far:

Method	Parameter(s)
ILS	Perturbation strength/depth of search
TS	Tabu tenure (list length)
LA	List length (L)
SA	$T_0, (T_\delta / \alpha / \beta)$
GD	T_{final}

- Many other metaheuristics/hyper-heuristics with many more parameters you will see over the coming lectures.



Types of Parameters

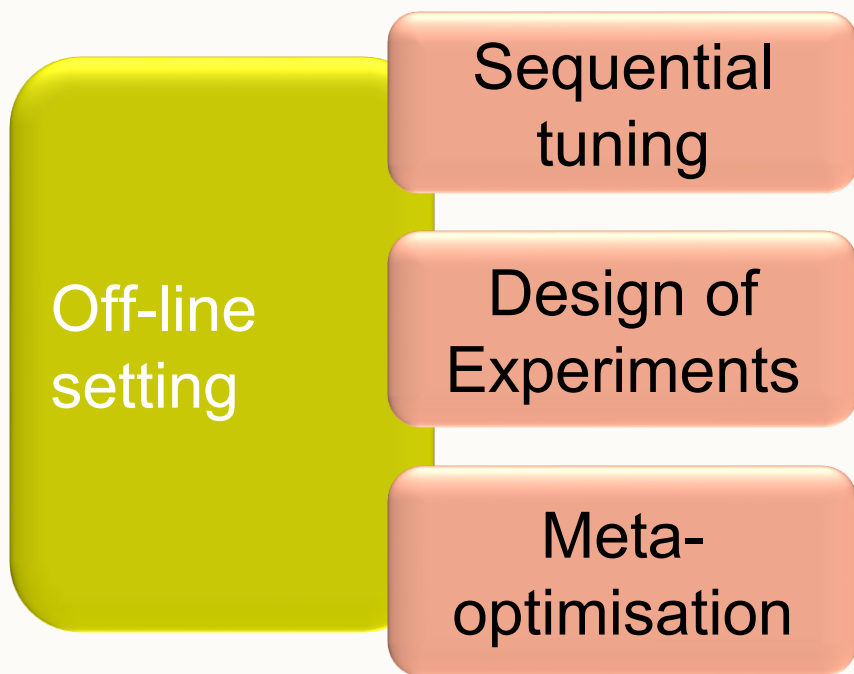
- Categorical/symbolic/structural parameters
 - Choice of initialisation method, choice of mutation,...
- Ordinal parameters
 - Neighbourhoods (e.g., small, medium, large),...
- **Numerical/behavioural parameters**
 - Integer, real-valued, ...
 - Population sizes, evaporation rates,...
 - Values may depend on the setting of categorical or ordinal parameters



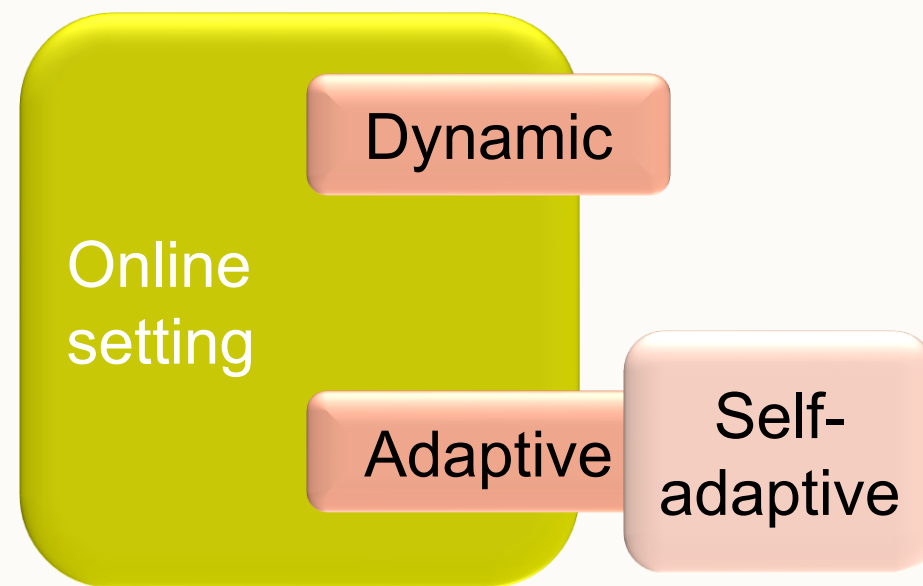
Parameter Setting Methods – Tuning

- **Parameter tuning:** Find the best initial settings for a set of parameters before the search process starts (*off-line*). E.g., fixing the mutation strength in ILS, mutation probability in genetic algorithms, etc.

Parameter Tuning



Parameter Control

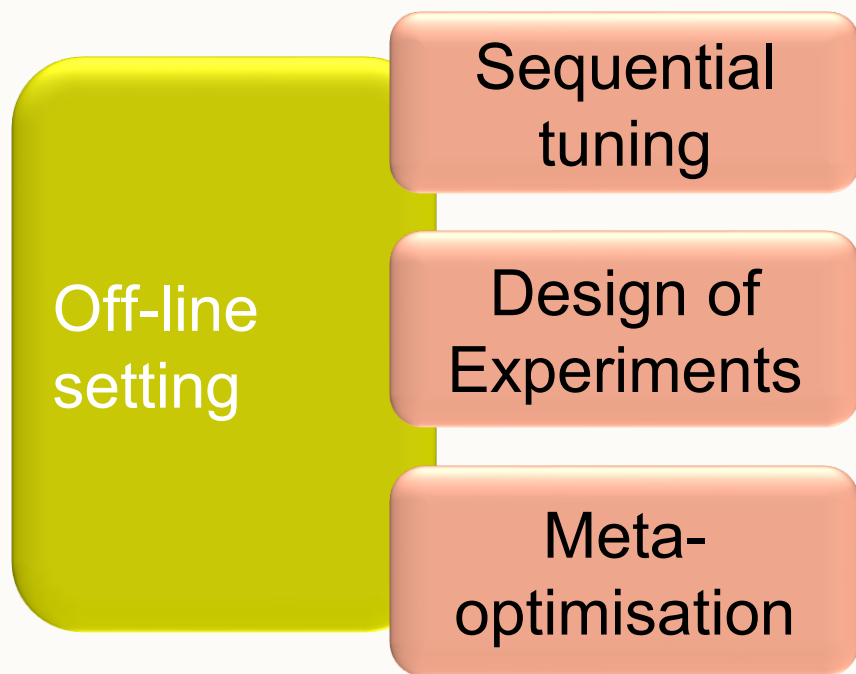




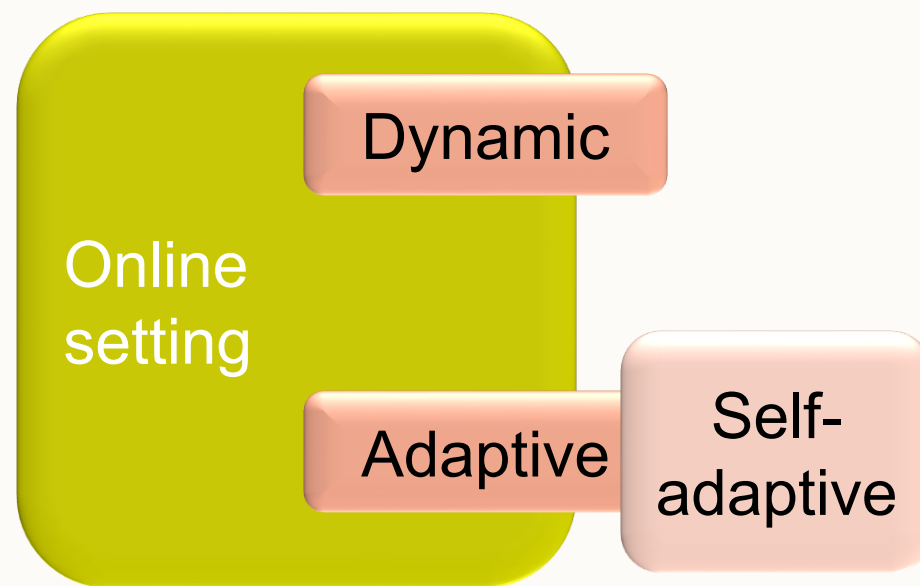
Parameter Setting Methods – Control

- **Parameter control:** Adapt the settings of parameters during the search process (*online*) (dynamic, adaptive, self-adaptive). E.g., changing the mutation strength in ILS, changing the mutation probability in genetic algorithms during the search process.

Parameter Tuning



Parameter Control





Parameter Tuning Methods (1)

- Assume that we have a metaheuristic with two parameters, and we have discretised some options:
 - $\alpha \in \{0.0, 0.3, 0.5, 0.8, 1.0\}$, and
 - $\beta \in \{20, 40, 60, 80, 100\}$
- Simple tuning approaches include:
 - **Arbitrary choice** (randomly pick $\alpha = 0.3$ and $\beta = 60$).
 - **Trial and error** – possibly with some intuition.
 - Suggested settings from theoretical studies.
 - A combination of the above.

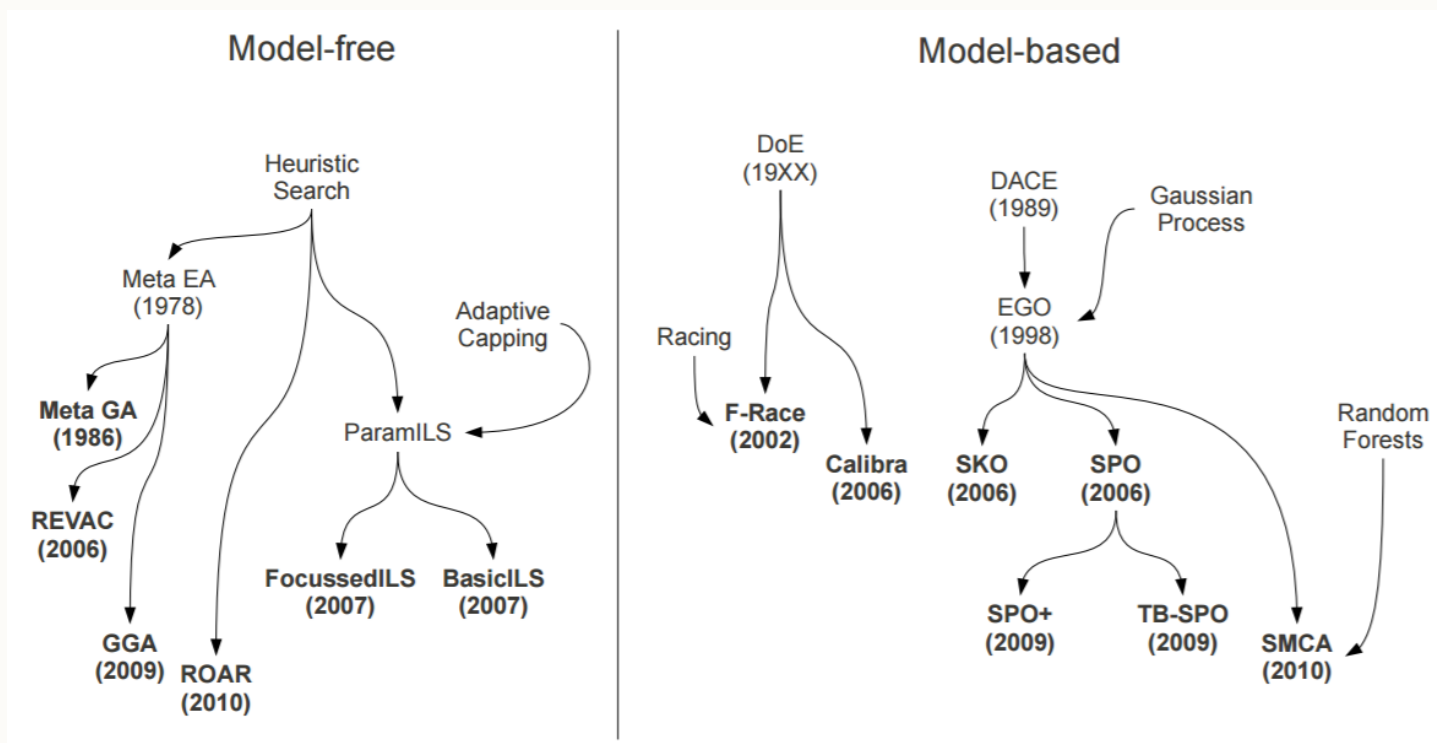


Parameter Tuning Methods (2)

- Assume that we have a metaheuristic with two parameters, and we have discretised some options:
 - $\alpha \in \{0.0, 0.3, 0.5, 0.8, 1.0\}$, and
 - $\beta \in \{20, 40, 60, 80, 100\}$
- **Sequential tuning** involves fixing all but one parameter and trying each setting successively.
 - Example: pick random values for $\alpha = 0.3, \beta = 60$
 - Try all configurations of α with β fixed to 60.
 - Using the best setting for α , now try configurations for β .
- Design of experiments are systematic methods (examples later).
- Meta-optimisation.



Automated Parameter Tuning Methods



Source: http://www.mff.cuni.cz/veda/konference/wds/proc/pdf10/WDS10_109_i1_Dobslaw.pdf

I-race (download: <http://iridia.ulb.ac.be/irace/>)

ParamILS (download: <http://www.cs.ubc.ca/labs/beta/Projects/ParamILS/>)

SPOT (download: <https://cran.r-project.org/web/packages/SPOT/index.html>)



Design of Experiments (DoE)

- A systematic method (controlled experiments) to determine the relationship between controllable and uncontrollable *factors* (inputs to the process, variables) affecting a process (e.g., running of an algorithm), their *levels* (settings) and the *response* (output) of that process (e.g., quality of solutions obtained – performance of an algorithm). (Fisher 1926, 1935)
- Important outcomes are measured and analysed to determine the factors and their settings that will provide the best overall outcome.
 - E.g., Two factors; $A \in \{0.0, 0.3, 0.5, 0.8, 1.0\}$ and $B \in \{20, 40, 60, 80, 100\}$, each with 5 levels – at least 5^2 runs required.



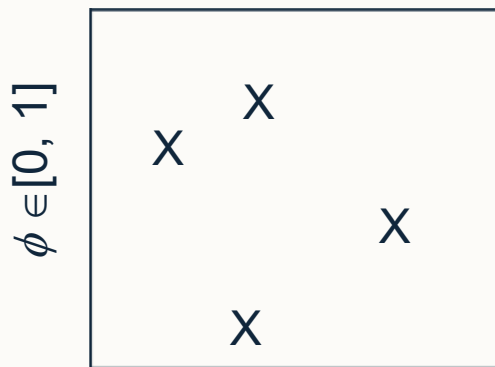
Fractional Factorial Designs

- Imagine we have k factors in an n level factorial design, then we have n^k designs.
- **Fractional factorial designs** reduce the number of required configurations by systematically sampling the parameter space to draw valuable conclusions.
- Consider we have two parameters and 8 different settings for each; we would need to evaluate $8^2 = 64$ configurations.
- Fractional designs can significantly reduce this, e.g. only 8 (n).



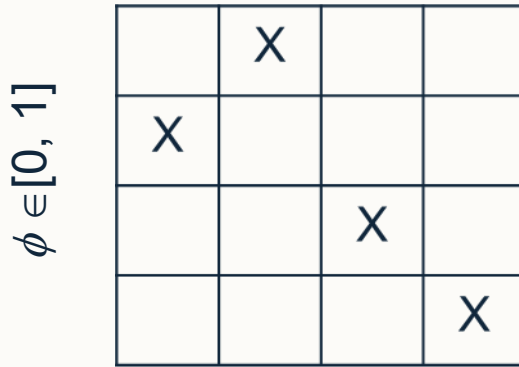
DoE Sampling Methods

- There are three main sampling methods used for parameter tuning.
- **Random** – choose ‘m’ random samples/configurations.
- **Latin Hyper-cube** – choose ‘m’ samples ensuring that specific settings are not repeated.
- **Orthogonal** – ensures a better spread in the sample by subdividing the sample space into equally probably subspaces.



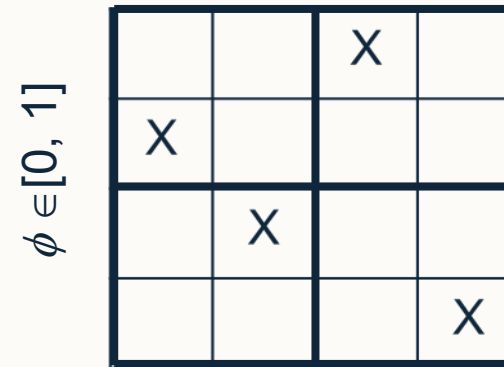
$\delta \in [0.2, 0.8]$

Random



$\delta \in [0.2, 0.8]$

Latin Hypercube



$\delta \in [0.2, 0.8]$

Orthogonal



Parameter Tuning

A Case Study using the Taguchi Method



The Taguchi Orthogonal Arrays Method

- Developed by Genichi Taguchi to improve the quality of goods by making the manufacture of a product/process less variable (more robust).
- The Taguchi method is a structured statistical (DoE) method for determining the best combination of parameter settings to achieve a certain objective.
- Also shown to be useful in configuring the settings of meta/hyper-heuristics.



Taguchi Orthogonal Arrays

- Generally works well when there are 3 to 50 (parameters/variables/factors) and when there are few interactions between variables and only a few variables contribute significantly to the overall performance.
- **Taguchi Orthogonal Arrays** are highly fractional orthogonal designs and can be used to estimate **main effects** using only a few experimental runs.

L4 (2^3)

Run	Columns		
	1	2	3
1	1	1	1
2	1	2	2
3	2	1	2
4	2	2	1

L8 (2^7)

Run	Columns						
	1	2	3	4	5	6	7
1	1	1	1	1	1	1	1
2	1	1	1	2	2	2	2
3	1	2	2	1	1	2	2
4	1	2	2	2	2	1	1
5	2	1	2	1	2	1	2
6	2	1	2	2	1	2	1
7	2	2	1	1	2	2	1
8	2	2	1	2	1	1	2

L9 (3^4)

Run	Columns			
	1	2	3	4
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	2	1	2	3
5	2	2	3	1
6	2	3	1	2
7	3	1	3	2
8	3	2	1	3
9	3	3	2	1



Overview – How to use the Taguchi Method for Parameter Tuning

- **Planning stage**

1. Select control parameters (factors)
2. Select possible settings for each parameter (levels)
3. Select the appropriate orthogonal array.

- **Experiment stage**

4. Conduct the experiments with the parameter configurations from (3) on a subset of “training”/test instances.

- **Analysis stage**

5. Analyse the results.
6. Determine the best levels for each of the parameters.

- **Validation stage**

7. Run confirmation experiment with the best reported configuration.



Step 1 – Parameter Tuning using Taguchi OAs

- In the planning stage step 1 we need to consider the parameters of our metaheuristic. Assume that we have:
 - $\chi_1 \in [0.0, 1.0] : \mathbb{R}$
 - $\chi_2 \in [0.0, 1.0] : \mathbb{R}$
 - $\chi_3 \in [1..80] : \mathbb{Z}^+$
 - $\chi_4 \in [1..5] : \mathbb{Z}^+$



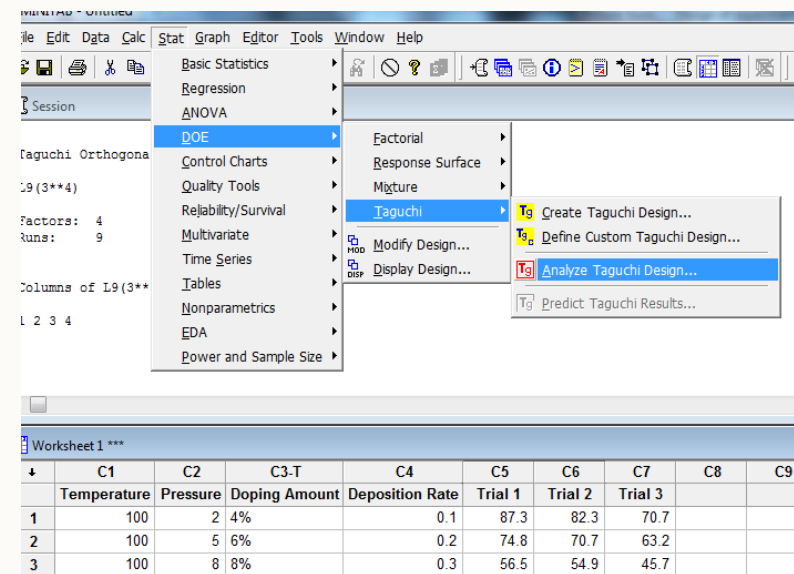
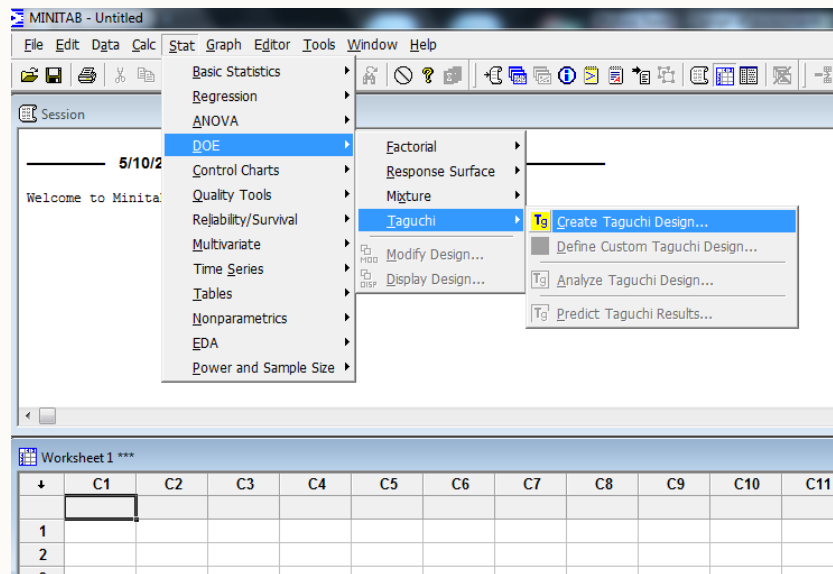
Step 2 – Parameter Tuning using Taguchi OAs

- In the planning stage step 2, we need to choose different settings for each of the parameters to try out (levels). Assume that we chose:
 - $\chi_1 \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ [5 levels]
 - $\chi_2 \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$ [5 levels]
 - $\chi_3 \in \{5, 10, 20, 40, 80\}$ [5 levels]
 - $\chi_4 \in \{2, 3, 4, 5\}$ [4 levels]
- We have a 4 factor, 5 level DoE.



Step 3a – Parameter Tuning using Taguchi OAs

- In the planning stage step 3, we need to choose the appropriate orthogonal array.
- Many packages exist which generate these for you including Minitab, MATLAB, R, Octave, JMP, DOE++.



Minitab



Step 3b – Parameter Tuning using Taguchi OAs

- Select the 4-factor and 5-level design corresponding to a (L25) array.
- Note OA designs can be found by an online search engine – further details can be found in [Kacker et al \(1991\)](#)

Parameters	Value Options
Intensity of Mutation (IoM)	{ 0.2 , 0.4, 0.6, 0.8, 1.0}
Depth of Search (DoS)	{0.2, 0.4, 0.6, 0.8, 1.0}
Population size (PopSize)	{5, 10, 20, 40, 80}
Tournament size (TourSize)	{2, 3, 4, 5}

Taguchi orthogonal array

Experiment number	IoM	DoS	Pop size	Tour size
1	1	1	1	1
2	1	2	2	2
3	1	3	3	3
4	1	4	4	4
5	1	5	5	2
6	2	1	2	3
7	2	2	3	4
8	2	3	4	1
9	2	4	5	1
10	2	5	1	2
11	3	1	3	4
12	3	2	4	1
13	3	3	5	2
14	3	4	1	3
15	3	5	2	4
16	4	1	4	2
17	4	2	5	3
18	4	3	1	4
19	4	4	2	2
20	4	5	3	1
21	5	1	5	4
22	5	2	1	3
23	5	3	2	1
24	5	4	3	2
25	5	5	4	3



Step 4 – Parameter Tuning using Taguchi OAs

- Now we need to conduct the experiments using the parameter configurations from the L25 array.
- **Following good practice**, we should run each configuration for multiple trials (≥ 30) and potentially across multiple problems/instances.

Experiment number	param1	param2	param3	param4
1	0.2	0.2	5	2
2	0.2	0.4	10	3
3	0.2	0.6	20	4
4	0.2	0.8	40	5
5	0.2	1.0	80	3
6	0.4	0.2	10	4
7	0.4	0.4	20	5
8	0.4	0.6	40	2
9	0.4	0.8	80	2
10	0.4	1.0	5	3
11	0.6	0.2	20	5
12	0.6	0.4	40	2
13	0.6	0.6	80	3
14	0.6	0.8	5	4
15	0.6	1.0	10	5
16	0.8	0.2	40	3
17	0.8	0.4	80	4
18	0.8	0.6	5	5
19	0.8	0.8	10	3
20	0.8	1.0	20	2
21	1.0	0.2	80	5
22	1.0	0.4	5	4
23	1.0	0.6	10	2
24	1.0	0.8	20	3
25	1.0	1.0	40	4



Step 5 – Parameter Tuning using Taguchi OAs

- Need a **performance metric** to compare across more than one instance such as Formula 1 scoring.
 - Best algorithm/configuration scores 10.
 - Next best scores 8.
 - Then 6, 5, 4, 3, 2, 1.
 - Scores for ties are shared evenly.
- Sum F1 score across all instances and report in the final column.

Experiment number	param1	param2	param3	param4	Total F1 score
1	0.2	0.2	5	2	24.88
2	0.2	0.4	10	3	14
3	0.2	0.6	20	4	17.50
4	0.2	0.8	40	5	17.88
5	0.2	1.0	80	3	15.50
6	0.4	0.2	10	4	9.88
7	0.4	0.4	20	5	2.50
8	0.4	0.6	40	2	8.88
9	0.4	0.8	80	2	22.50
10	0.4	1.0	5	3	27.25
11	0.6	0.2	20	5	0
12	0.6	0.4	40	2	1
13	0.6	0.6	80	3	12
14	0.6	0.8	5	4	24.38
15	0.6	1.0	10	5	22.25
16	0.8	0.2	40	3	0
17	0.8	0.4	80	4	25.88
18	0.8	0.6	5	5	30.88
19	0.8	0.8	10	3	16.25
20	0.8	1.0	20	2	18.38
21	1.0	0.2	80	5	4
22	1.0	0.4	5	4	0
23	1.0	0.6	10	2	1.88
24	1.0	0.8	20	3	1.88
25	1.0	1.0	40	4	2.50



Which configuration will perform the best for solving this problem instance?

A. $x_1=0.6$; $x_2=0.2$; $x_3=20$; $x_4=5$

B. $x_1=0.8$; $x_2=0.6$; $x_3=5$; $x_4=5$

✓ C. None of the above

Experiment number	param1	param2	param3	param4	mean F1 score
1	0.2	0.2	5	2	24.88
2	0.2	0.4	10	3	14
3	0.2	0.6	20	4	17.50
4	0.2	0.8	40	5	17.88
5	0.2	1.0	80	3	15.50
6	0.4	0.2	10	4	9.88
7	0.4	0.4	20	5	2.50
8	0.4	0.6	40	2	8.88
9	0.4	0.8	80	2	22.50
10	0.4	1.0	5	3	27.25
11	0.6	0.2	20	5	0
12	0.6	0.4	40	2	1
13	0.6	0.6	80	3	12
14	0.6	0.8	5	4	24.38
15	0.6	1.0	10	5	22.25
16	0.8	0.2	40	3	0
17	0.8	0.4	80	4	25.88
18	0.8	0.6	5	5	30.88
19	0.8	0.8	10	3	16.25
20	0.8	1.0	20	2	18.38
21	1.0	0.2	80	5	4
22	1.0	0.4	5	4	0
23	1.0	0.6	10	2	1.88
24	1.0	0.8	20	3	1.88
25	1.0	1.0	40	4	2.50

EchoPoll stopped working at this point hence there are not poll responses to report on. During the lecture, most students voted for “B” which is incorrect as we need to look into the mean effects plots (next slide) and perform the validation stage.

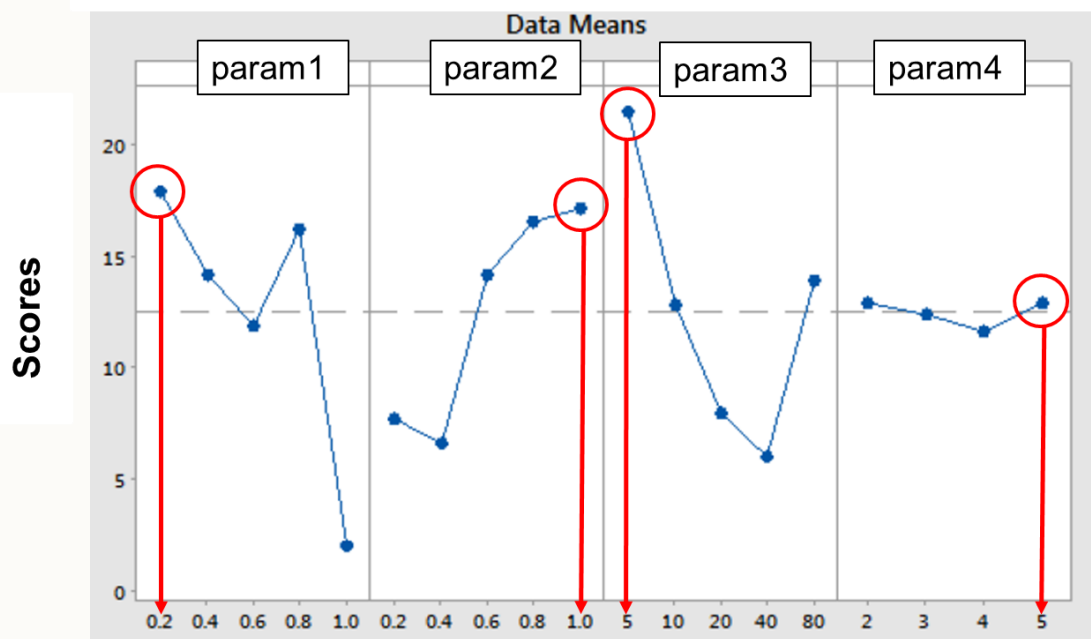


Step 6 – Parameter Tuning using Taguchi OAs

- Perform analysis with **main effects** as mean of F1 scores for each parameter level.

- E.g. $\chi_3 = 20$:

$$\text{Take } \frac{(17.5+2.5+0+18.38+1.88)}{5} = 8.052$$

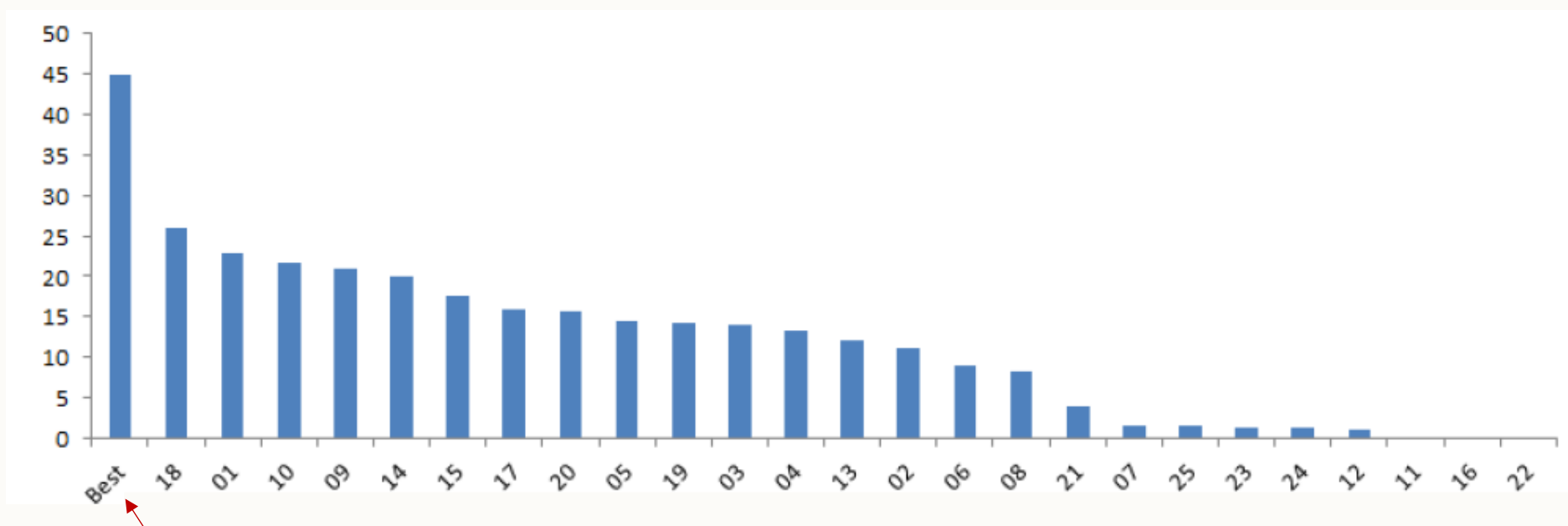


Experiment number	param1	param2	param3	param4	mean F1 score
1	0.2	0.2	5	2	24.88
2	0.2	0.4	10	3	14
3	0.2	0.6	20	4	17.50
4	0.2	0.8	40	5	17.88
5	0.2	1.0	80	3	15.50
6	0.4	0.2	10	4	9.88
7	0.4	0.4	20	5	2.50
8	0.4	0.6	40	2	8.88
9	0.4	0.8	80	2	22.50
10	0.4	1.0	5	3	27.25
11	0.6	0.2	20	5	0
12	0.6	0.4	40	2	1
13	0.6	0.6	80	3	12
14	0.6	0.8	5	4	24.38
15	0.6	1.0	10	5	22.25
16	0.8	0.2	40	3	0
17	0.8	0.4	80	4	25.88
18	0.8	0.6	5	5	30.88
19	0.8	0.8	10	3	16.25
20	0.8	1.0	20	2	18.38
21	1.0	0.2	80	5	4
22	1.0	0.4	5	4	0
23	1.0	0.6	10	2	1.88
24	1.0	0.8	20	3	1.88
25	1.0	1.0	40	4	2.50



Step 7 – Parameter Tuning using Taguchi OAs

- Finally, we need to confirm/validate that the given configuration is actually the best, comparing to the trialled configurations.
- If it is not, there could be a problematic choice of levels/dependence between selected parameters.



New configuration from mean effects plot.



Concluding remarks

- Module activities:
 - Formative self-assessment quiz on Moodle.
 - Computing Session (COMP2001): Friday 20th February CS-A32 4pm – Metaheuristics (Part 2).
 - **Next lecture with Ender:** 24th February 4pm – Evolutionary Algorithms (Part I) – Population-based Metaheuristics.



University of
Nottingham

UK | CHINA | MALAYSIA

Thank you

Any questions?